
Part 2: CSS



Introduction

- ▶ CSS = **Cascading Style Sheets**
 - ▶ A styling language
 - ▶ Current version: CSS3
- ▶ **Cascading**: the procedure that determines which style will apply to a certain section, if you have more than one style rule
- ▶ **Style**: how you want a certain part of your page to look
- ▶ **Sheets**: sets of rules for determining how the webpage will look

Principle

- ▶ A style rule has 2 parts:
 - ▶ **Selector**: the HTML elements you want to add style to
 - ▶ **Declaration**: the statement of style for that element, made up of property and value
- ▶ Ex:
 - ▶

```
selector {  
    property1: value1;  
    property2: value2;  
    // ...  
}
```
- ▶ The selector can either be a grouping of elements, an identifier, class, or single HTML element

Inline = chỉnh 1 dòng

Internal = chỉnh 1 trang

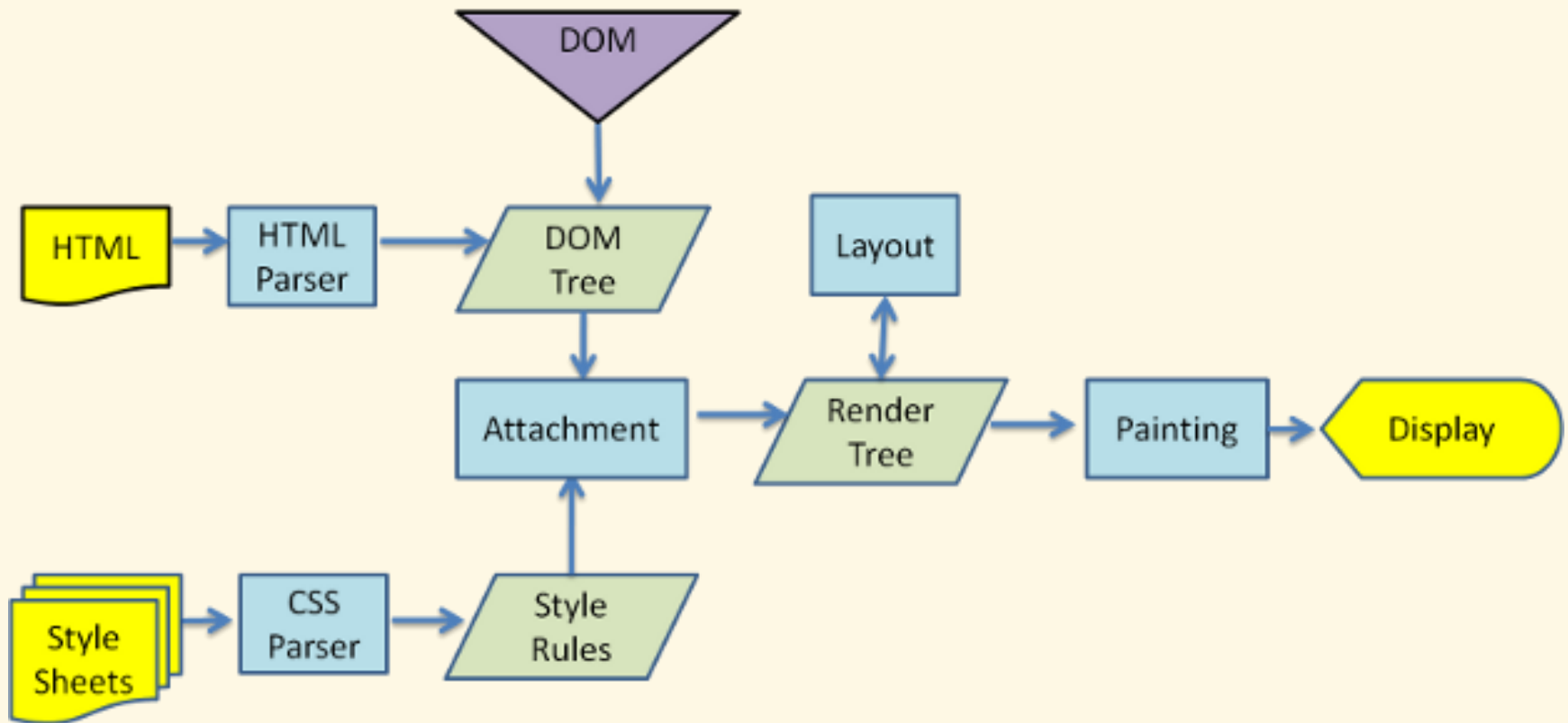
External = chỉnh cả website

Attaching a Style Sheet

@import = CSS gọi CSS

- ▶ Attach a style sheet to a HTML page by adding the code to the `<head>` section. There are 4 ways to do.
- ▶ **Inline Style Sheet:** Used directly within HTML tags
 - ▶ `<p style="color: red">Some Text</p>`
- ▶ **Internal Style Sheet:** Best used to control styling on one page
 - ▶ `<style type="text/css">`
`h1 {color: red}`
`</style>`
- ▶ **External Style Sheet:** Best used to control styling on multiple pages
 - ▶ `<link rel="stylesheet" type="text/css"`
`media="all" href="css/styles.css" />`
- ▶ **Inside another CSS file:**
 - ▶ `@import url(coolblue.css);`

Rendering Engine Flow



Selectors

Element/Tag Selector

- ▶ Specify the style(s) for a single HTML element type
 - ▶ `body` {
 margin: 0;
 padding: 0;
 border-top: 1px solid #ff0;
}
- ▶ Grouping elements: allows you to specify a single style for multiple elements at one time
 - ▶ `h1, h2, h3, h4, h5, h6` {
 font-family: "Trebuchet MS", sans-serif;
}

Identifier (ID) Selector

- ▶ Specify the style(s) for element identified by its ID
 - ▶ `<p id="first-intro">Introduction text</p>`
 - ▶ `#first-intro {
border-bottom: 2px dashed #fff;
}`

Class Selector

- ▶ Specify the style(s) for element(s) of user-defined classes
 - ▶ `<p class="intro">Introduction text</p>`
 - ▶ `.intro {
 font: 12px verdana, sans-serif;
 margin: 10px;
}`
- ▶ An element can use multiple classes:
 - ▶ `<p class="intro primary animated">...</p>`
- ▶ Identifier or class?
 - ▶ An identifier is specified only once on a page and has a higher inheritance specificity than a class
 - ▶ A class is reusable as many times as needed in a page
 - ▶ Use identifiers for main sections and sub-sections of your document

Element with Class or ID Selector

- ▶ Specify the style(s) for elements of a type, and identified by a class or ID at the same time
 - ▶ `p.intro { }`
 - ▶ `p#first-intro { }`
 - ▶ `p.intro#first-intro { }`
 - ▶ `p.intro.primary { }`
 - ▶ `p.intro.primary#first-intro { }`
 - ▶ `.intro#first-intro { }`

Child Selector

- ▶ Specify the style(s) for child element(s) of another

- ▶ `.intro > h1 {`
 `font-size: 2rem;`
 `}`
 - ▶ `#navigation > p {`
 `line-height: 20px;`
 `}`

Descendant Selector

- ▶ Specify the style(s) for descendant element(s) of another

- ▶ `body h1 {`
 `font-size: 2rem;`
 `}`
- ▶ `#navigation p {`
 `line-height: 20px;`
 `}`

Adjacent Sibling Selector

- ▶ Specify the style(s) for element that is directly after another

- ▶ `p.intro + h1 {`
 `font-size: 2rem;`
 `}`
- ▶ `#navigation + p {`
 `line-height: 20px;`
 `}`

General Sibling Selector

- ▶ Specify the style(s) for sibling element(s) of another

- ▶ `p.intro ~ h1 {`
 `font-size: 2rem;`
 `}`
- ▶ `#navigation ~ p {`
 `line-height: 20px;`
 `}`

Attribute Selector

- ▶ Specify the style(s) for element(s) with the specified attribute and value
 - ▶ Elements with `target` attribute
 - ▶ `a[target] { }`
 - ▶ Elements whose `target` attribute value is “_blank”
 - ▶ `a[target="_blank"] { }`
 - ▶ Elements whose `title` attribute value contains “flower”
 - ▶ `a[title~="flower"] { }` (value needs be whole word)
 - ▶ `a[title*="flower"] { }` (value doesn't need to be whole word)
 - ▶ Elements whose `title` attribute value begins with “flower”
 - ▶ `a[title^="flower"] { }`
 - ▶ Elements whose `title` attribute value ends with “flower”
 - ▶ `a[title$="flower"] { }`

Universal Selector

- ▶ Specify the style(s) for all elements

- ▶ `* {`
 `color: gray;`
 `}`
 - ▶ `#navigation * {`
 `color: blue;`
 `}`

The Power of Cascade

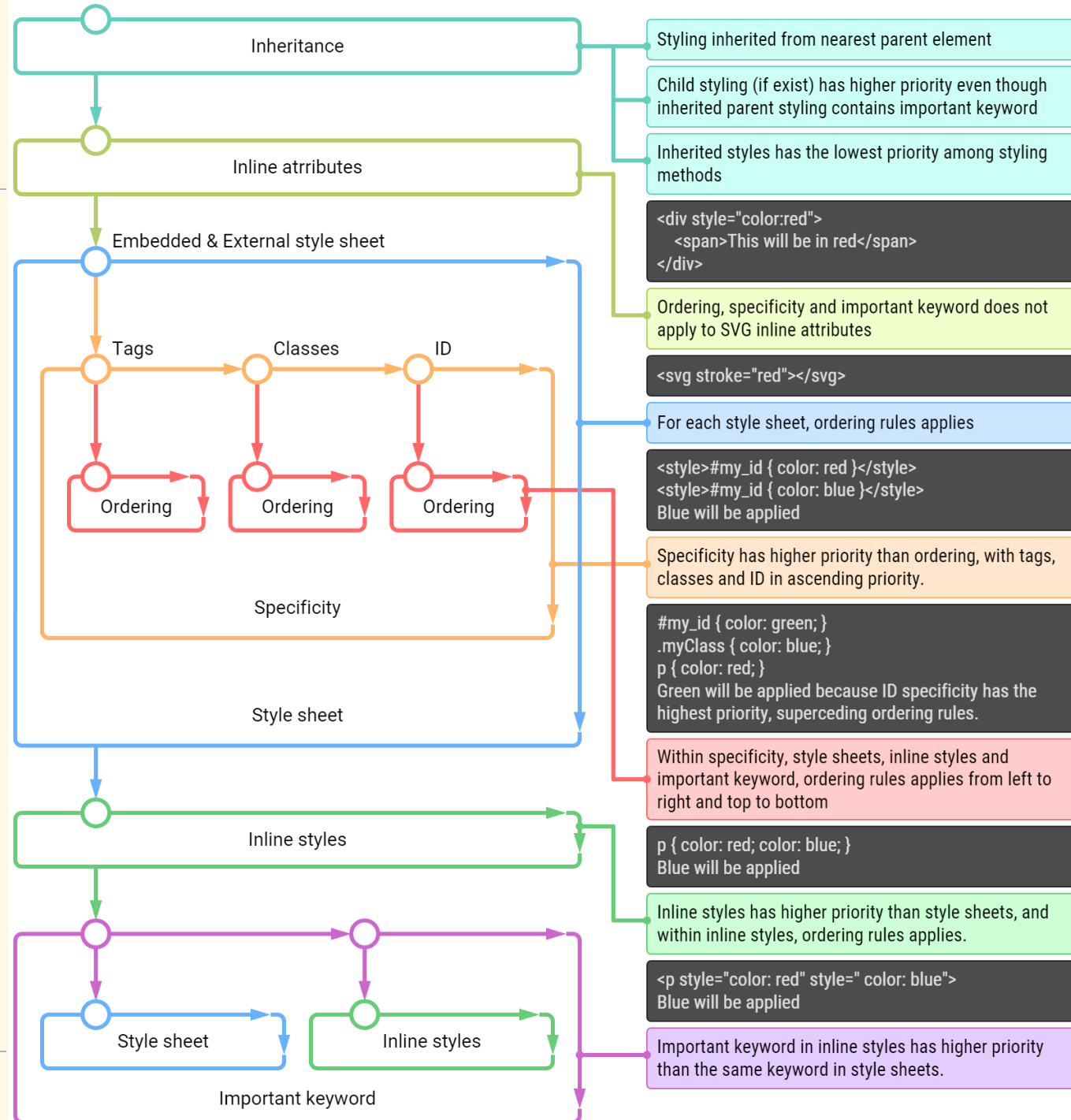
- ▶ When multiple styles or style sheets are used, they start to cascade and sometimes compete with one another due to CSS's inheritance feature
- ▶ Any tag on the page could potentially be affected by any of the tags surrounded by it
- ▶ So, which one wins? Nearest ancestor wins:
 - ▶ Inline style or directly applied style
 - ▶ The last style sheet declared in the `<head>` section

Inheritance

- ▶ Inheritance means CSS properties applied to one tag are passed on to nested tags
 - ▶ Only applied to properties whose value is `inherit`
 - ▶ `<body style="font-family: Arial">`
 `<p>This text will be Arial as well</p>`
 `</body>`
- ▶ Inherited property groups: font, spacing, text, list, visibility,...
- ▶ Non-inherited property groups: layout, box, background, alignment,...

Precedence

► By ascending order



Exercise

- Style tables as the followings. Pay attention to header row, odd/even rows, row on hover, “TYPE” column.

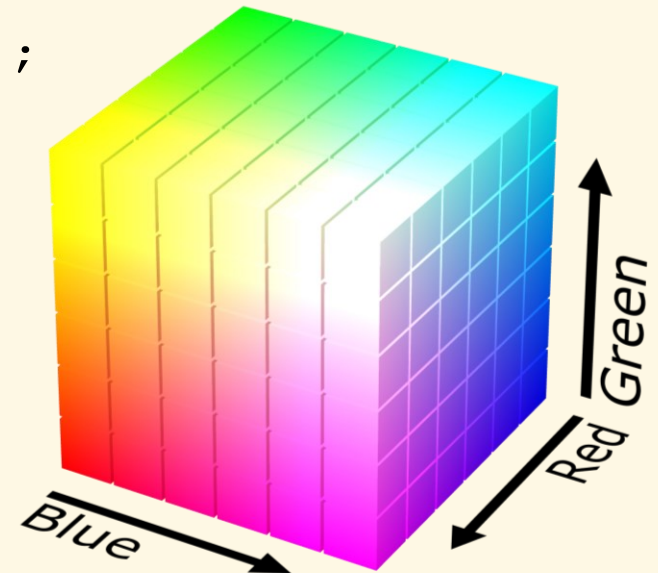
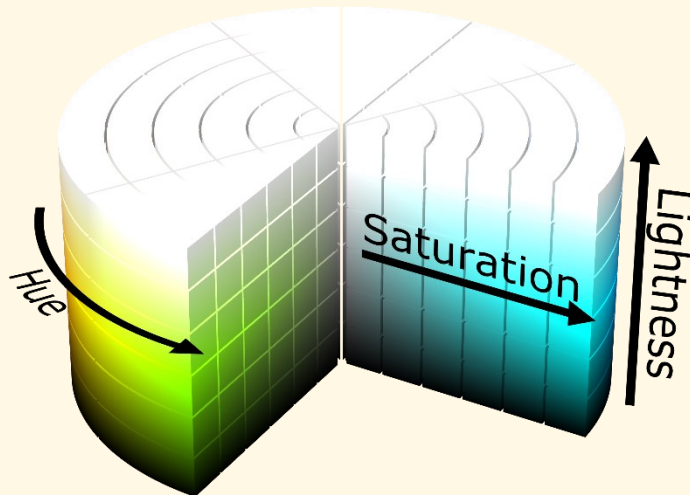
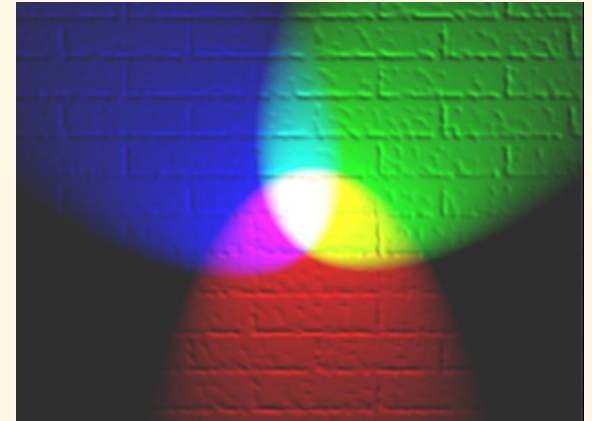
	TYPE	MESSAGE	USER
	cron	Cron run completed.	Anonymous (not verified)
	cron	Cron run completed.	Anonymous (not verified)
	system	views_ui module enabled.	admin
	system	views_ui module installed.	admin
	system	toolbar module enabled.	admin
	system	toolbar module installed.	admin
	svstem	breakpoint module	admin

Text Formatting

Text Color

▶ Give color values to color property

- ▶ `color: red;`
- ▶ `color: #0f0;`
- ▶ `color: #00ff00;`
- ▶ `color: #00ff0080;`
- ▶ `color: rgb(0, 127, 255);`
- ▶ `color: rgba(0, 127, 255, 0.5);`
- ▶ `color: hsl(120, 100%, 80%);`
- ▶ `color: hsla(120, 100%, 80%, 0.5);`



Text Properties

- ▶ `text-align: left|right|center|justify;`
- ▶ `vertical-align: top|bottom|middle|baseline;`
- ▶ `text-decoration: underline|overline|line-through;`
- ▶ `text-transform: uppercase|lowercase|capitalize;`

- ▶ `text-indent: 20px;`
- ▶ `letter-spacing: 5px;`
- ▶ `word-spacing: 15px;`
- ▶ `line-height: 1.2;`

Font Properties

- ▶ `font: 15px Arial, sans-serif;`
- ▶ `font: italic small-caps bold 12px/1.2 Times;`
- ▶ `font-style: normal|italic;`
- ▶ `font-variant: normal|small-caps;`
- ▶ `font-weight: normal|bold|bolder|lighter|200|400|700;`
- ▶ `font-size: xx-small|x-small|smaller|small|medium|large|15px, 1em, 1rem;`
- ▶ `line-height: 1.2;`
- ▶ `font-family: "Times New Roman", Times, serif;`

CSS Length Units

▶ Absolute units:

- ▶ mm, cm, in: millimeters, centimeters, inches
- ▶ pt: points (1pt = 1/72 of 1in)
- ▶ px: pixels
- ▶ pc: picas (1pc = 12pt)

▶ Relative units:

- ▶ em: relative to the font-size of the element (2em means 2 times the size of the current font)
- ▶ rem: relative to the font-size of the root element (<html> element)
- ▶ %: relative to the parent element
- ▶ vw, vh: percentage relative to the browser window

Background Styling

Background Color

- ▶ **Give color values to background-color property**
 - ▶ `background-color: red;`
 - ▶ `background-color: #00ff00;`
 - ▶ `background-color: #00ff0080;`
 - ▶ `background-color: rgb(0, 127, 255);`
 - ▶ `background-color: rgba(0, 127, 255, 0.5);`
 - ▶ `background-color: hsl(120, 100%, 80%);`
 - ▶ `background-color: hsla(120, 100%, 80%, 0.5);`

Background Image

- ▶ `background: url(leaves.jpg) no-repeat top left;`
- ▶ `background-image: url(leaves.jpg);`
- ▶ `background-repeat: no-repeat|repeat|repeat-x|repeat-y;`
- ▶ `background-position: left|right|center|20%|10px top|center|bottom|30%|20px;`
- ▶ `background-size: auto|cover|contain|50%;`
- ▶ `background-origin: padding-box|border-box|content-box;`
- ▶ `background-clip: border-box|padding-box|content-box;`
- ▶ `background-attachment: scroll|fixed|local;`

Multiple Background Images

- ▶ Possible to put multiple images together in one background
- ▶ `.multi-bg {
 background-image: url(decoration.png),
 url(ribbon.png), url(old_paper.jpg);
 background-repeat: no-repeat;
 background-position: left top,
 right bottom, left top;
}`



Inline Image

- ▶ Images can be embedded directly into CSS with Base64 encoding:
 - ▶ `background-image:`
`url (data:image/gif;base64,R0lGODlhEAAQAMQA`
`AORHHOVSKudfOulrSOp3WOyDZu6QdvCchPGolf00o/`
`XBs/fNwfjZ0frl3/zy7///wAAAAAAAAAAAAAAAAAAAA`
`AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA`
`AAACH5BAkAABAALAAAAAAQABAAAVVICSOZGlCQAos`
`J6mu7fiyZeKqNKToQGDsM8hBADgUXoGAiqhSvp5QAn`
`QKGIgUhwFUYLCVDFCrKUE1lBavAViFIDlTImbKC5Gm`
`2hB0SlBCBMQiB0UjIQA7) ;`

Image in Text Background

- ▶ Currently CSS has no properties for image in text background
- ▶ Use the following trick:

```
.text {  
    color: transparent;  
    background-image: url(leaves.png);  
    -webkit-background-clip: text;  
}
```



Text with Image
Background

- ▶ Attn: `background-clip` property is standard, but `text` value is not, hence use `-webkit-` prefix for WebKit-based browsers (Chrome, Edge, Safari,...)

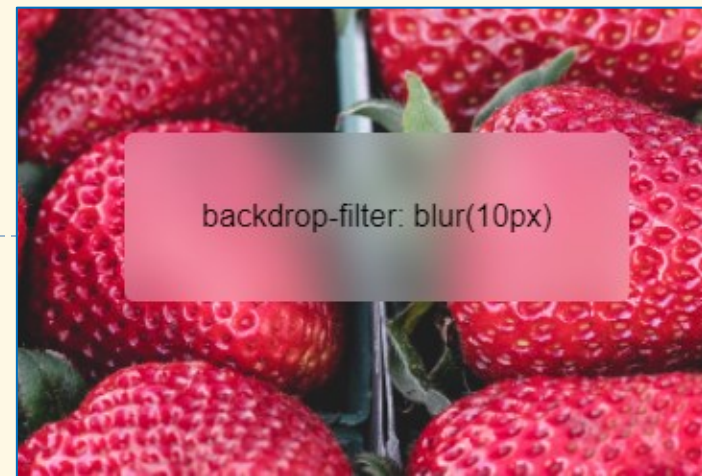
Backdrop as Background

- ▶ **With backdrop-filter property:**

- ▶ `backdrop-filter: blur(2px);`
- ▶ `backdrop-filter: contrast(40%);`
- ▶ `backdrop-filter: grayscale(30%);`
- ▶ `backdrop-filter: brightness(60%);`
- ▶ `backdrop-filter: drop-shadow(4px 4px 10px blue);`
- ▶ `backdrop-filter: hue-rotate(120deg);`
- ▶ `backdrop-filter: invert(70%);`
- ▶ `backdrop-filter: opacity(20%);`
- ▶ `backdrop-filter: sepia(90%);`
- ▶ `backdrop-filter: saturate(80%);`

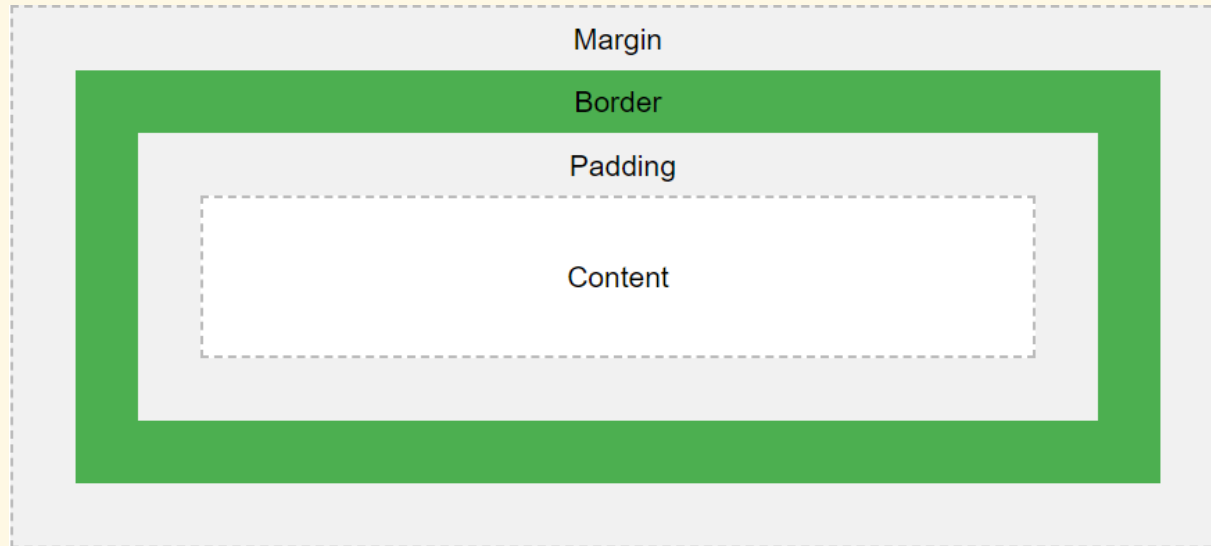
- ▶ **Multiple filters possible:**

- ▶ `backdrop-filter: blur(4px) saturate(150%);`



Box Model

Introduction



- ▶ All HTML elements can be considered as boxes
- ▶ The box model is a box consisting of:
 - ▶ Margin: Clears an area outside the border, and is transparent.
 - ▶ Border: Goes around the padding and content.
 - ▶ Padding: Clears an area around the content, and is transparent.
 - ▶ Content: Where text and child elements appear.

Box Sizing Modes

- ▶ `box-sizing: content-box | border-box;`
- ▶ `content-box`:
 - ▶ Default mode
 - ▶ Width and height properties include only the content
 - ▶ Actual size (w/h) = content + paddings + borders + margins
 - ▶ width, height = content size
- ▶ `border-box`:
 - ▶ Width and height properties include content, padding and border
 - ▶ Actual size (w/h) = width/height + margins
 - ▶ width, height = content + paddings + borders

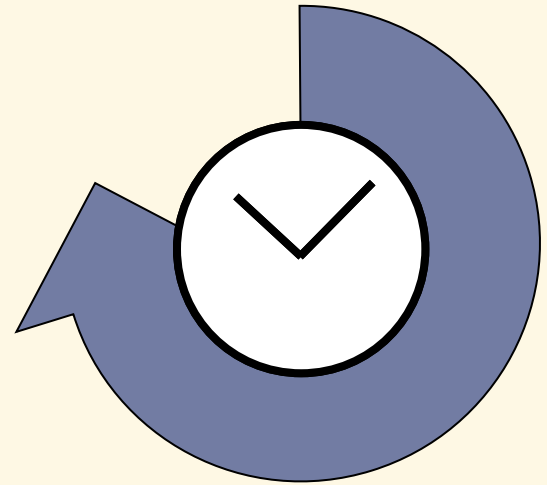
Width and Height Properties

- ▶ Size of elements are controlled by:
 - ▶ `width: 200px | 20em | 30rem | 40% | auto | fit-content;`
 - ▶ `height: 100px;`
 - ▶ `max-width, min-width: 500px | 80%;`
 - ▶ `max-height, min-height: 500px;`
- ▶ `calc()` function:
 - ▶ `width: calc(10px + 100px);`
 - ▶ `max-width: calc(100% - 30px);`
 - ▶ `height: calc(2em * 5);`
 - ▶ Make sure that operators (+, -, *, /) are surrounded by whitespaces

Paddings

- ▶ `padding: auto;`
- ▶ `padding: 5px;`
- ▶ `padding: 10px 2rem;`
- ▶ `padding: 10px 0 1.2em 1rem;`

- ▶ `padding-top: 2px;`
- ▶ `padding-left: 2rem;`



Margins

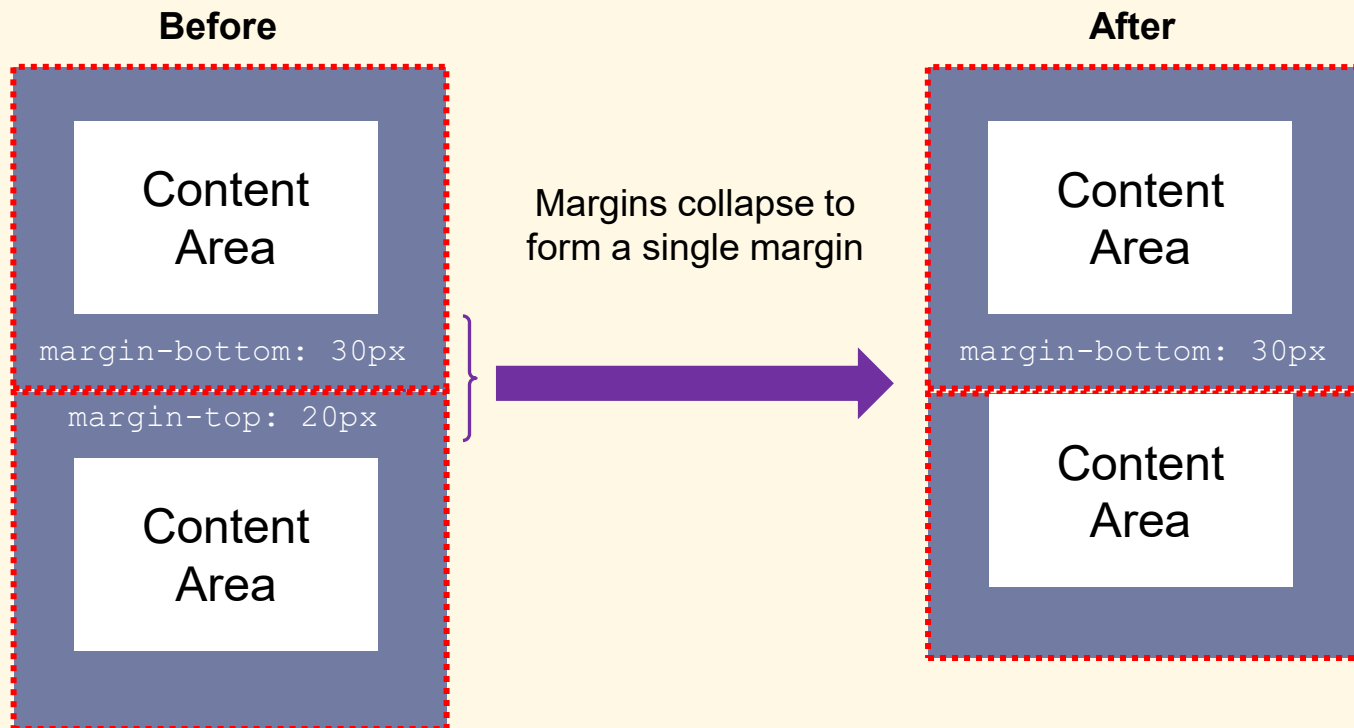
- ▶ `margin: auto;`
- ▶ `margin: 5px;`
- ▶ `margin: 10px 2rem;`
- ▶ `margin: 10px 0 1.2em 1rem;`

- ▶ `margin-top: 2px;`
- ▶ `margin-left: 2rem;`

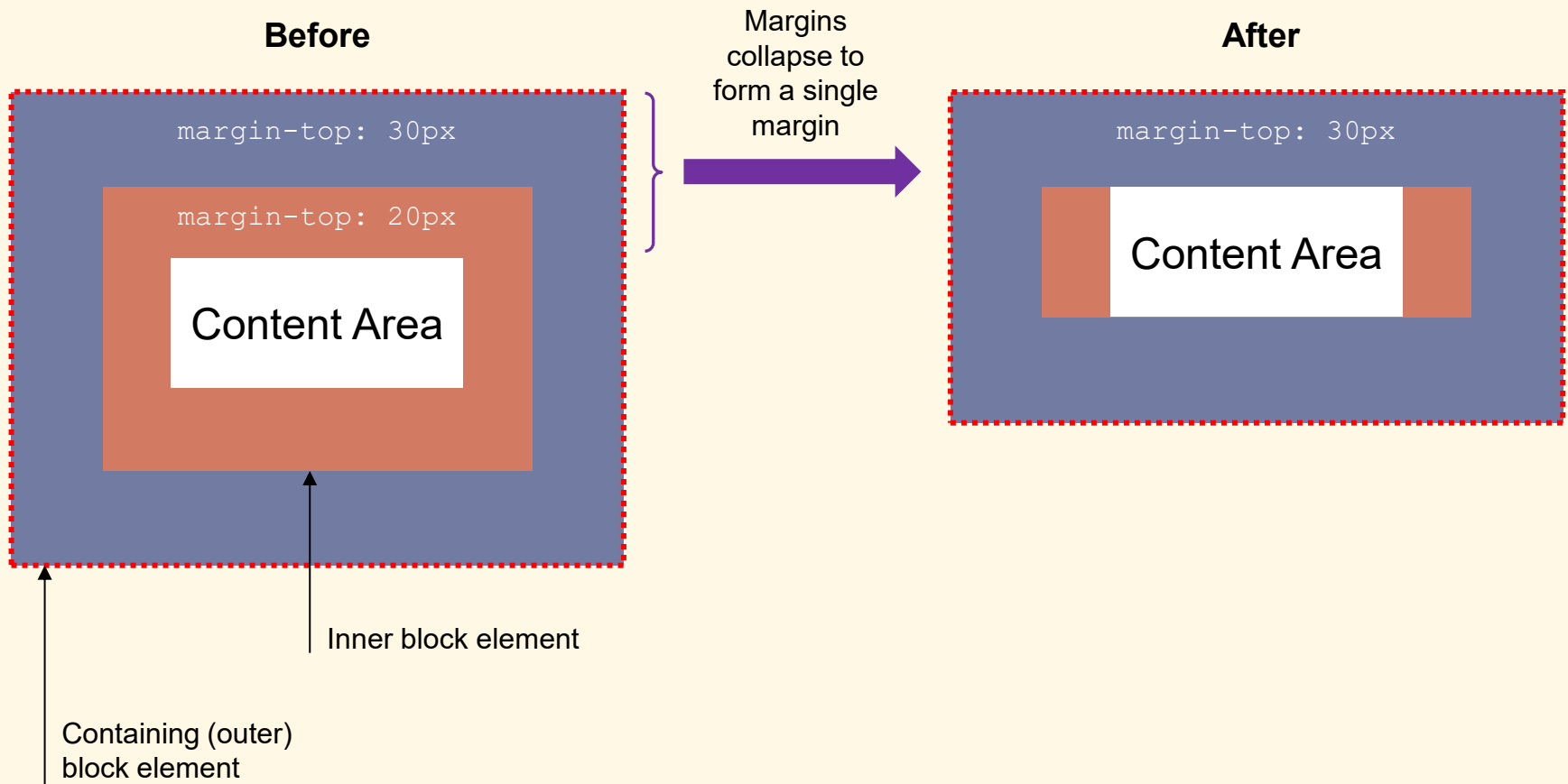
Margin Collapse

- ▶ When 2 or more vertical margins meet, they will collapse to form a single margin
 - ▶ The height of this combined margin is the largest of them
 - ▶ Only happens if there are no borders or padding separating the margins
- ▶ Margin collapse applies in 2 cases
 - ▶ 2 or more block elements are stacked one above the other
 - ▶ One block element is contained within another block element

Margin Collapse for Stacked Elements

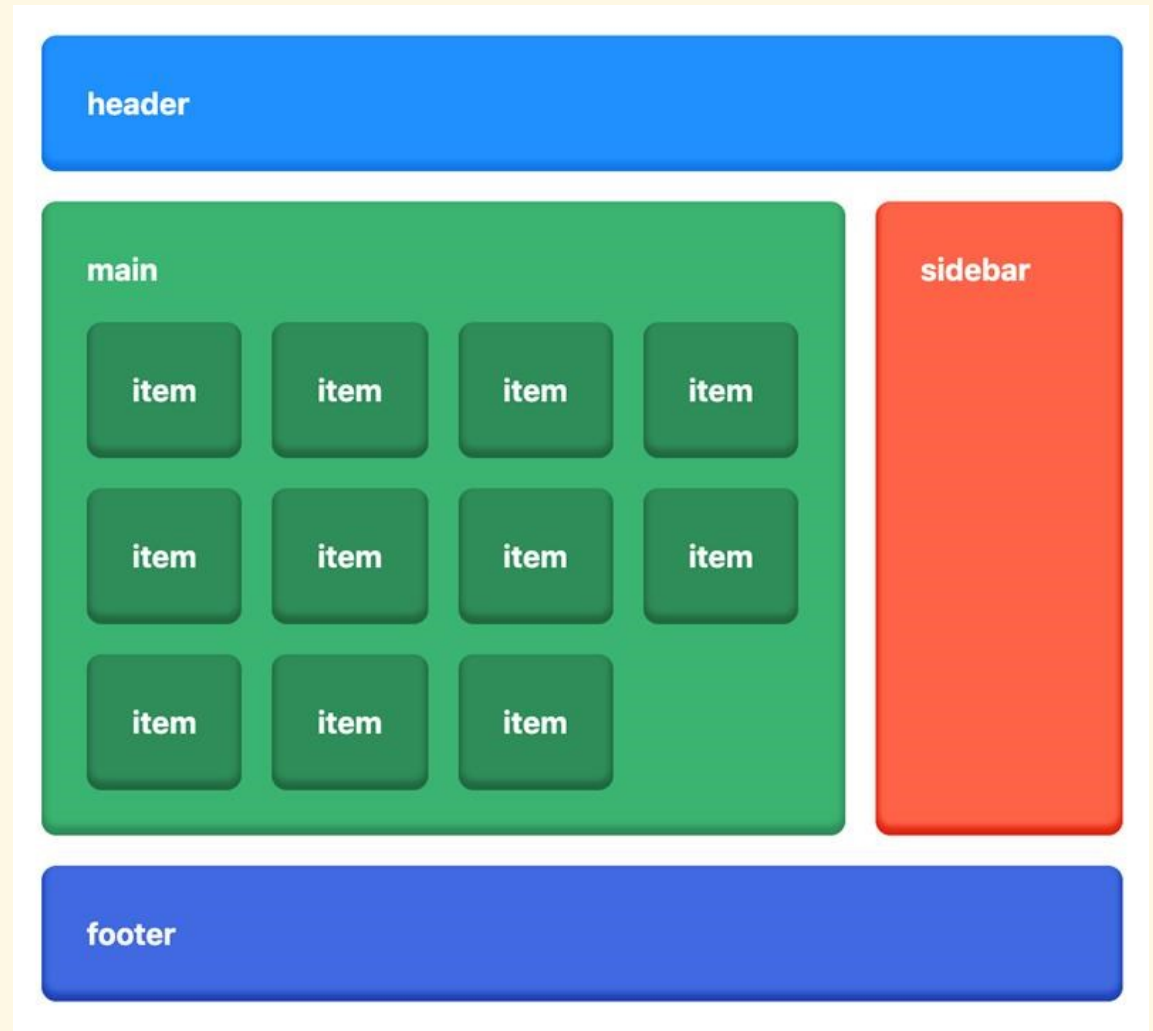


Margin Collapse for Contained Elements



Exercise

- ▶ Create a layout as in the figure using only `<div>` elements



Border Styling

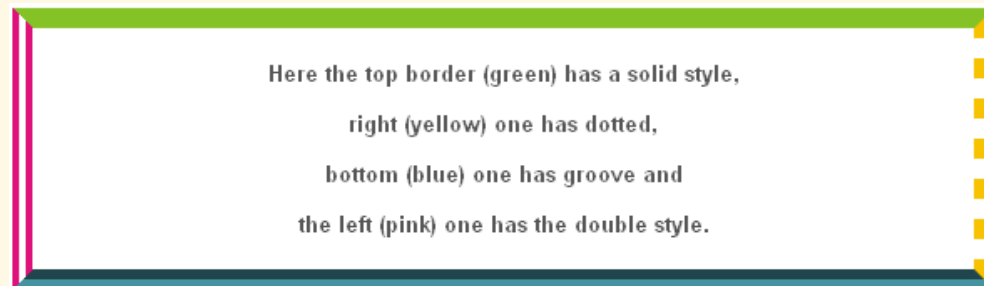
Border Color

- ▶ `border-color: red;`
- ▶ `border-color: red #f29 rgb(0, 127, 255) blue;`
- ▶ `border-color: red blue;`

- ▶ `border-top-color: #00ff0080;`
- ▶ `border-right-color: rgba(0, 127, 255, 0.5);`
- ▶ `border-bottom-color: rgb(0, 127, 255);`
- ▶ `border-left-color: #00ff00;`

Border Style

- ▶ `border-style: solid|dotted|dashed|double|groove|inset|outset;`
- ▶ `border-style: solid dotted groove double;`
- ▶ `border-style: solid dashed;`
- ▶ `border-left-style: solid;`



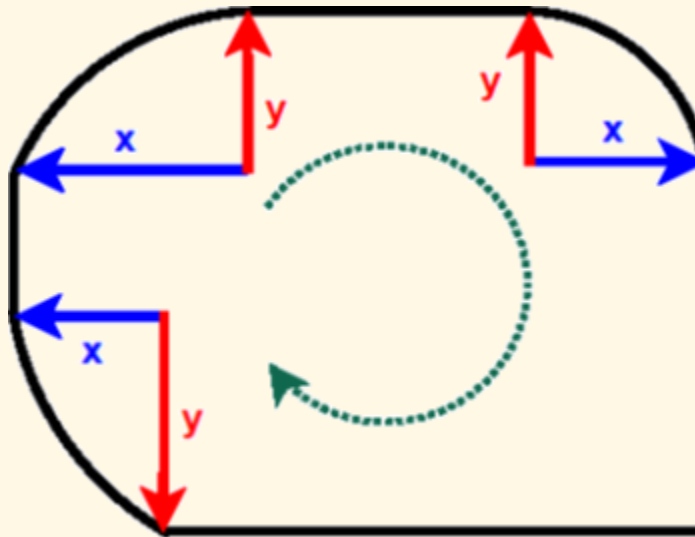
- ▶ `border-width: 2px|medium|thick|thin;`
- ▶ `border-width: 1px 0 5px thin;`
- ▶ `border-width: 2px thin;`
- ▶ `border-right-width: 3px;`

Border Shorthands

- ▶ `border: 5px solid red;`
- ▶ `border-top: 2px dotted #25f;`

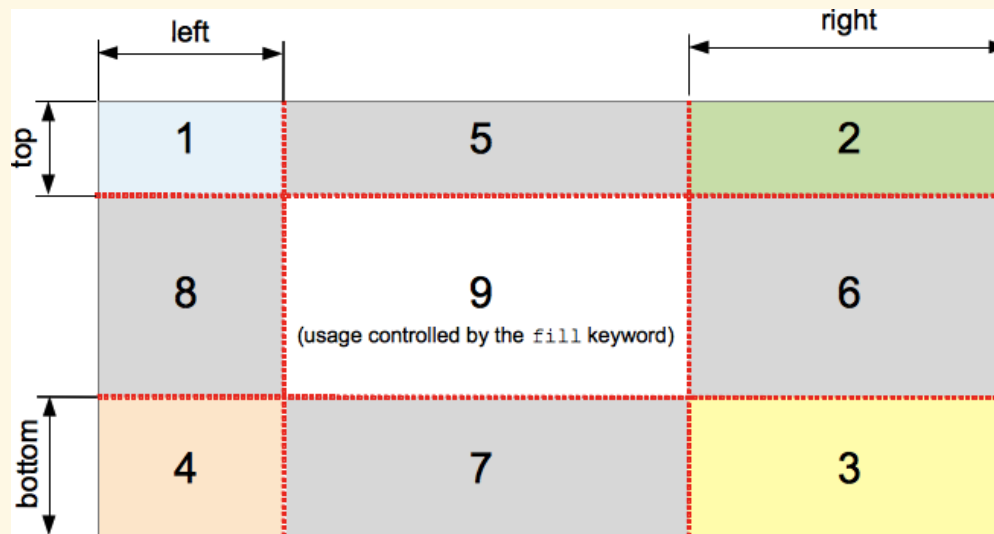
Rounded Corners

- ▶ `border-radius: 5px|30%;`
- ▶ `border-radius: 2px 1rem 10% 3em;`
- ▶ `border-radius: 2px 3rem 0 4em / 1rem 2px 0 10%;`
- ▶ `border-top-left-radius: 10px;`
- ▶ `border-top-right-radius: 10px 30%;`



Border Image

- ▶ `border-image: url(border.png) 30 round;`
- ▶ `border-image-source: url(border.png);`
- ▶ `border-image-slice: 30 20% fill 10;`



- ▶ `border-image-width: 10px;`
- ▶ `border-image-repeat: repeat|round|stretch|space;`

Outline

- ▶ An outline is a line that is drawn just outside the border edge of the elements
 - ▶ Generally used to indicate focus or active states of the elements such as buttons, links, form fields
 - ▶ Outlines don't take up space, may overlap other elements
 - ▶ Unlike borders, an outline is the same on all sides
 - ▶ Outlines may be non-rectangular, but you cannot create circular outlines
- ▶ `outline: 2px dotted green;`
- ▶ `outline-width: 1px|thick|thin|medium;`
- ▶ `outline-style: solid|dotted|dashed|ridge;`
- ▶ `outline-color: gray;`
- ▶ `outline-offset: 10px;`

Positioning & Display

Positioning Methods

- ▶ `position` property:
 - ▶ `static` (default): Elements render in order, as they appear in the document flow
 - ▶ `absolute`: Element is positioned relative to its first positioned (not `static`) ancestor element
 - ▶ `fixed`: Element is positioned relative to the browser window
 - ▶ `relative`: Element is positioned relative to its normal position
 - ▶ `sticky`: Element is positioned based on the user's scroll position
- ▶ `position: absolute | relative | fixed | sticky` (except `static`) elements are called positioned

position: static

- ▶ `div.info {
 position: static;
 border: 3px solid #73AD21;
}`
- ▶ `<div class="info">
 This div element has position: static;
</div>`

Static position demo

An element with `position: static;` is not positioned in any special way; it is always positioned according to the normal flow of the page:

This div element has `position: static;`

position: relative

- ▶ `div.info {
 position: relative;
 left: 30px;
 border: 3px solid #73AD21;
}`
- ▶ `<div class="info">`
 This div element has position: relative;
`</div>`

Relative position demo

An element with `position: relative;` is positioned relative to its normal position:

`This div element has position: relative;`

position: fixed

- ▶ `div.info {`
 `position: fixed;`
 `top: 10px;`
 `right: 20px;`
 `width: 300px;`
 `border: 3px solid #73AD21;`
 `}`
- ▶ `<div class="info">`
 This div element has position: fixed;
 `</div>`

Fixed position demo
An element with position: fixed; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled:

position: absolute

```
▶ div.parent {  
    position: relative;  
    width: 400px;  
    height: 200px;  
    border: 3px solid #73AD21;  
}  
div.child {  
    position: absolute;  
    top: 80px;  
    right: 0;  
    width: 200px;  
    height: 100px;  
    border: 3px solid #73AD21;  
}
```

```
▶ <div class="parent">This element is relative;  
    <div class="child">This element is absolute;</div>  
</div>
```

Absolute position demo

An element with `position: absolute;` is positioned relative to the nearest positioned ancestor (instead of positioned relative to the viewport, like `fixed`):

This element is relative;

This element is absolute;

position: sticky

- ▶

```
div.info {  
    position: sticky;  
    top: 0;  
    padding: 5px;  
    background-color: #cae8ca;  
    border: 2px solid #4CAF50;  
}
```
- ▶

```
<div class="info">  
    This element is sticky!  
</div>
```

This element is sticky!

In this example, the sticky element sticks to the top of the page (top: 0), when you reach its scroll position.

Scroll back up to remove the stickyness.

Some text to enable scrolling.. Lorem ipsum dolor sit amet, illum definitiones no quo, maluisset concludaturque et eum, altera fabulas ut quo. Atqui causae gloriatur ius te, id agam omnis evertitur eum. Affert laboramus repudiandae nec et. Inciderint efficiantur his ad. Eum no molestiae voluptatibus.

z-Index

- ▶ `z-index` property specifies the stack order of an element
 - ▶ Element with greater stack order is always in front of an element with a lower stack order
 - ▶ Only works on positioned elements

```
▶ img {  
    position: absolute;  
    left: 0px;  
    top: -20px;  
    z-index: -1;  
}
```

```
▶ <div style="position: relative">  
    <h1>The z-index Property</h1>  
      
</div>
```

Because the image has a z-index of -1, it will be placed behind the heading.



Display Types

- ▶ **display property:**
 - ▶ **inline:** Element is inline (like `<span>`), any width and height properties have no effect
 - ▶ **block:** Element starts a new line and takes up the whole width (like `<div>`)
 - ▶ **inline-block:** Element is inline, but can take width and height properties
 - ▶ **none:** Element is not displayed
 - ▶ **list-item:** Element behaves like a `<li>`
 - ▶ **grid:** Element is a block-level grid container (discussed later)
 - ▶ **flex:** Element is a block-level flex container (discussed later)
 - ▶ **table, table-cell, table-row, table-column,...**

Visibility Methods

- ▶ `visibility` property:
 - ▶ `visible`: Element is displayed
 - ▶ `hidden`: Element is hidden
- ▶ “`visibility: hidden`” vs “`display: none`”:
 - ▶ `visibility: hidden`
 - ▶ Takes up space in the layout
 - ▶ Children are still visible if they have “`visibility: visible`”
 - ▶ `display: none`
 - ▶ Does not take up space
 - ▶ Removes the element completely, including all children

Float Methods

- ▶ Make elements float to one side of its container

- ▶ `float: left|right;`

- ▶ Example:

- ▶

```
img {  
    float: right;  
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum, nisi lorem egestas odio, vitae scelerisque enim ligula venenatis dolor. Maecenas nisl est, ultrices nec congue eget, auctor vitae massa. Fusce luctus vestibulum augue ut aliquet. Mauris ante ligula, facilisis sed ornare eu, lobortis in odio. Praesent convallis urna a lacus interdum ut hendrerit risus congue. Nunc sagittis dictum nisi, sed ullamcorper ipsum dignissim ac...



Clearing Floats

- ▶ Avoid floating elements on one or both sides
 - ▶ `clear: left | right | both;`

Without clear

div1

This text is intentionally on the right of div1.

div2 - Notice that div2 is after div1 in the HTML code. However, since div1 floats to the left, the text in div2 flows around div1.

With clear

div3

This text is intentionally on the right of div3.

div4 - Here, `clear: left;` moves div4 down below the floating div3. The value "left" clears elements floated to the left. You can also clear "right" and "both".

Clearfix

- ▶ Problem: If an element is taller than the element containing it, and it is floated, it will “overflow” outside of its container
- ▶ Solution: add “`overflow:auto`” to the containing element

Without Clearfix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



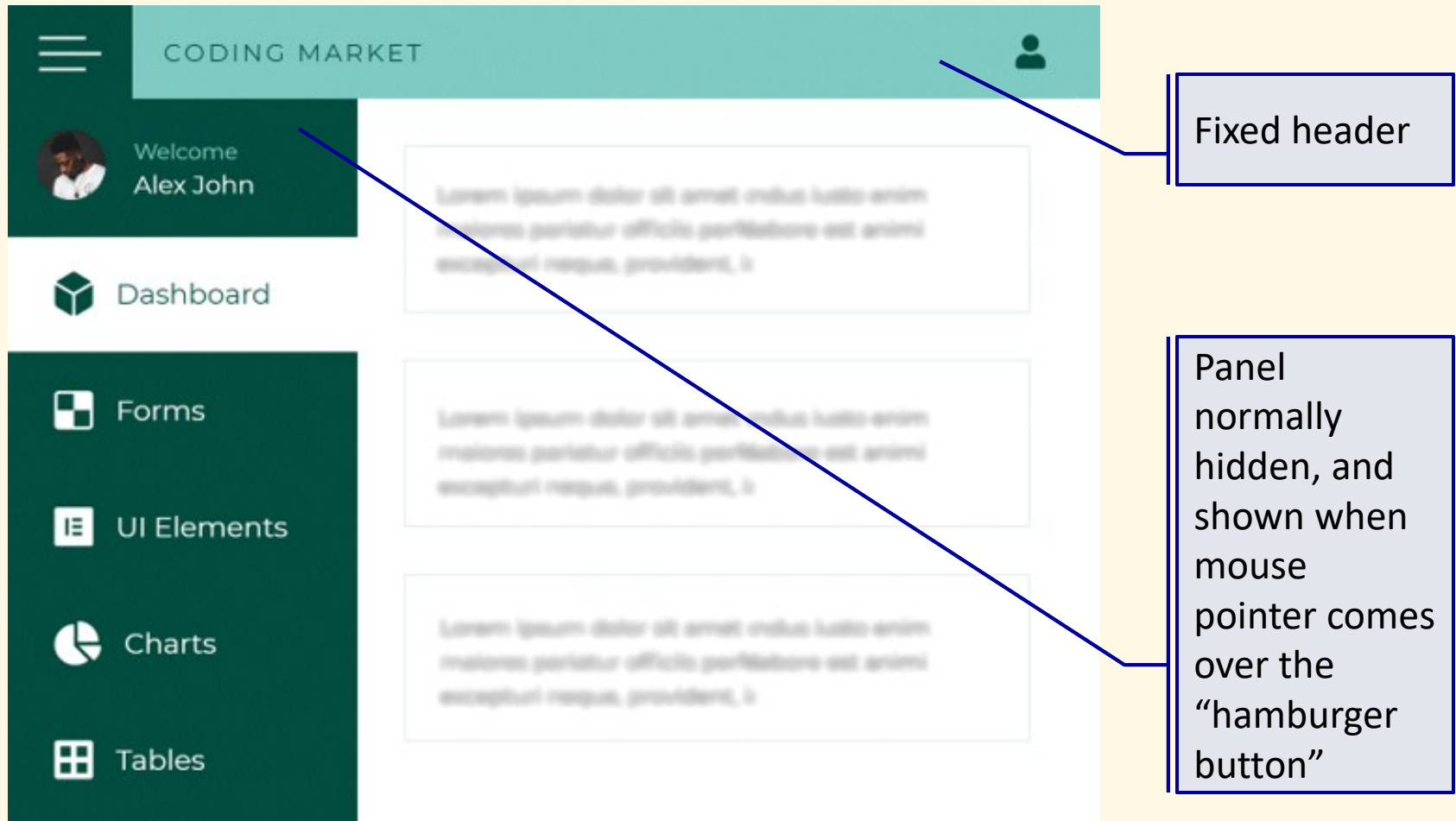
With Clearfix

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus imperdiet, nulla et dictum interdum...



Exercise

- Create a page with the following layout



Pseudo-classes

Introduction

- ▶ Pseudo-classes select regular elements but under certain conditions
- ▶ Types of pseudo-classes:
 - ▶ Dynamic pseudo-classes
 - ▶ State pseudo-classes
 - ▶ Range pseudo-classes
 - ▶ Structural pseudo-classes
 - ▶ Matching pseudo-classes
 - ▶ Other pseudo-classes

Dynamic Pseudo-classes

- ▶ Unvisited link:
 - ▶ `a:link`
- ▶ Visited link:
 - ▶ `a:visited`
- ▶ Selected link:
 - ▶ `a:active`
- ▶ Mouse-over:
 - ▶ `:hover`
- ▶ Focused element:
 - ▶ `:focus`
 - ▶ `:focus-within`

State Pseudo-classes

- ▶ Form control states:

- ▶ `:disabled`
- ▶ `:enabled`

- ▶ `<input>` states:

- ▶ `:checked`
- ▶ `:read-only`
- ▶ `:read-write`
- ▶ `:required`
- ▶ `:optional`
- ▶ `:valid`
- ▶ `:invalid`

Range Pseudo-classes

- ▶ First child:
 - ▶ `:first-child`
- ▶ Last child:
 - ▶ `:last-child`
- ▶ n^{th} child:
 - ▶ `:nth-child( $n$  [of selector])`
 - ▶ `p:nth-child(5)`
 - ▶ `p:nth-child(2 of .highlighted)`
 - ▶ `p:nth-child(odd)`
 - ▶ `p:nth-child(even)`
 - ▶ `p:nth-child(2n+1)`
- ▶ n^{th} last child:
 - ▶ `:nth-last-child( $n$ )`
- ▶ n^{th} child of type:
 - ▶ `:nth-of-type( $n$ )`
- ▶ n^{th} last child of type:
 - ▶ `:nth-last-of-type( $n$ )`

Structural Pseudo-classes

- ▶ Only child:
 - ▶ `:only-child`
- ▶ Only of type:
 - ▶ `:only-of-type`
- ▶ Root:
 - ▶ `:root`
- ▶ Empty:
 - ▶ `:empty`
- ▶ Target element:
 - ▶ `:target`

Matching Pseudo-classes

- ▶ Negation: elements not matching the given selector

- ▶ `:not(.intro) { }`

- ▶ `.intro:not(:first-child) { }`

- ▶ `p:not(.intro):not([title^="flower"]) { }`

- ▶ Matches-any: elements matching one of the given selectors

- ▶ `:is(header, .intro, :first-child) a { }`

- ▶ `.button:is(a, button) { }`

- ▶ `p:is(.header, .footer):not(.dark) { }`

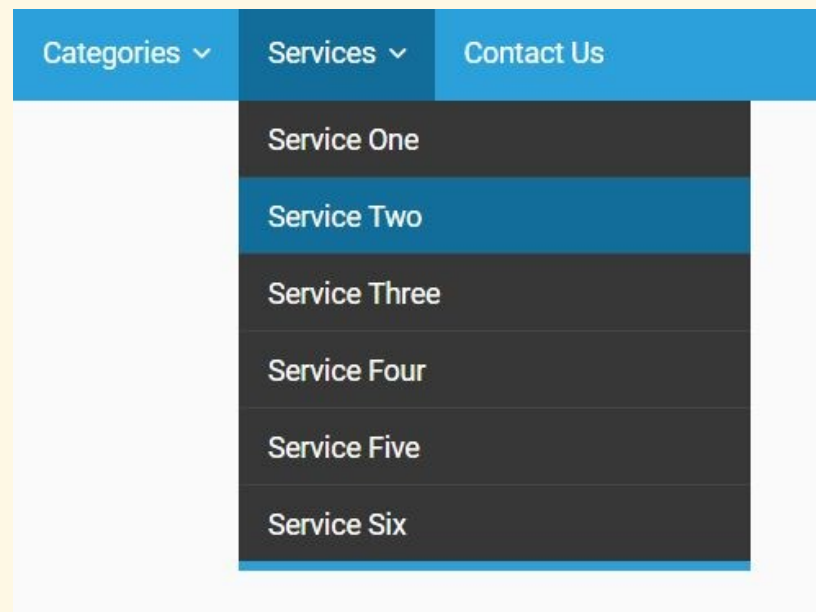
- ▶ Containment: elements containing specific children

- ▶ `h1:has(p) { }`

- ▶ `h1:has(h2):has(> p) { }`

Exercises

- ▶ Create a button group that looks like the figure
- ▶ Create a menubar with dropdown submenus



Pseudo-elements

Introduction

- ▶ Pseudo-elements effectively create new elements that are not specified in the markup of the document and can be manipulated much like a regular element
- ▶ Pseudo-elements:
 - ▶ `::before`
 - ▶ `::after`
 - ▶ `::first-letter`
 - ▶ `::first-line`

::before and ::after

- ▶ Insert some content before and after the content of the specified elements

```
▶ .intro::before {  
    content: "Read this:";  
    display: block;  
    color: green;  
}  
  
.intro::after {  
    content: "End of introduction";  
    display: block;  
    color: yellow;  
}
```

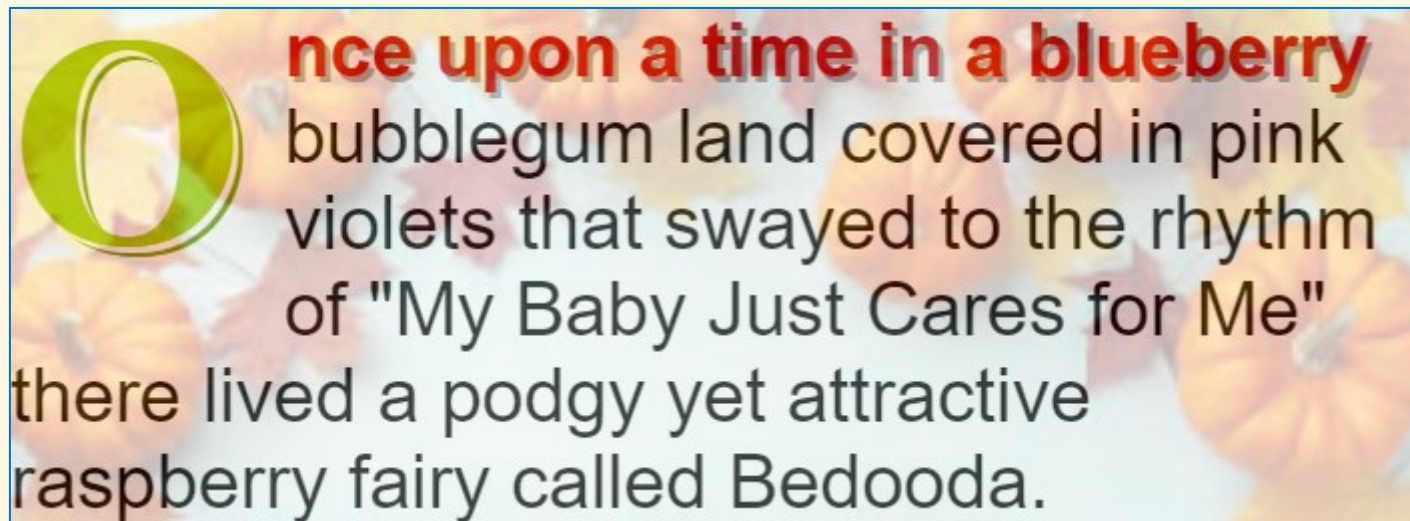
::first-letter and ::first-line

- ▶ Used to add a style to the first letter or the first line of the specified selector

```
▶ p::first-letter {  
    font-size: 200%;  
    color: #8A2BE2;  
}  
  
▶ p::first-line {  
    color: #ff0000;  
    font-variant: small-caps;  
}
```

Exercise

- Style a paragraph as in the following:

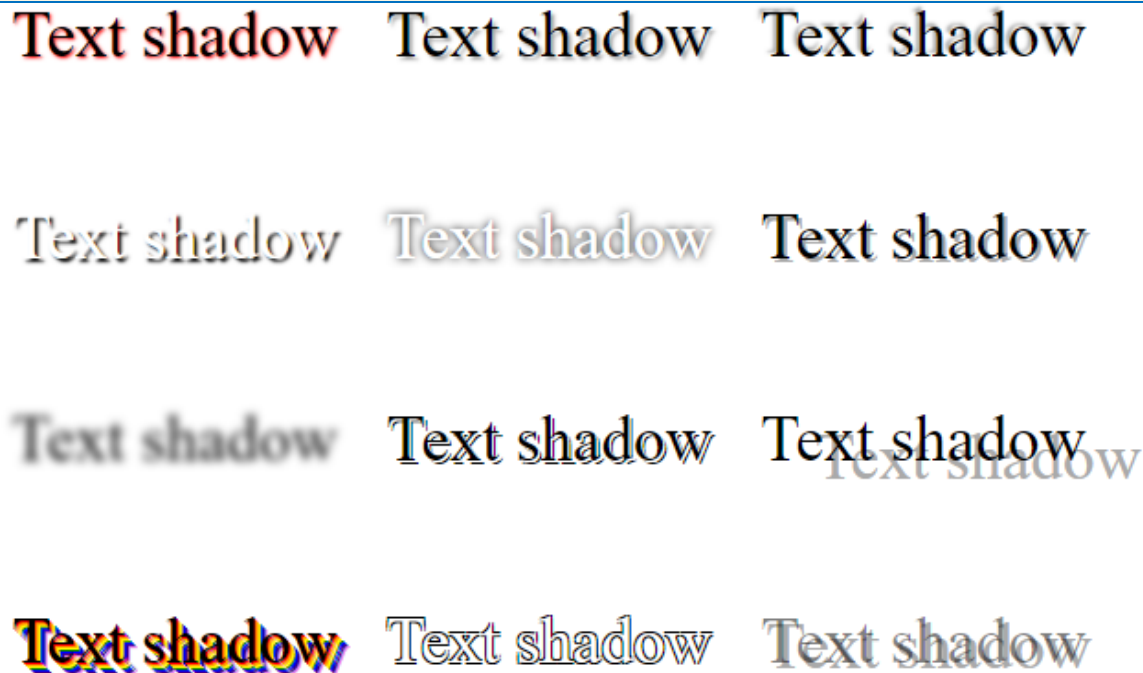


Effects

Text Shadow

► Syntax: `text-shadow: x y b c;`

- `x` = horizontal offset
 - `x < 0`: left of the text
 - `x > 0`: right of the text
- `y` = vertical offset
 - `y < 0`: above the text
 - `y > 0`: below the text
- `b` = blur radius
- `c` = shadow color



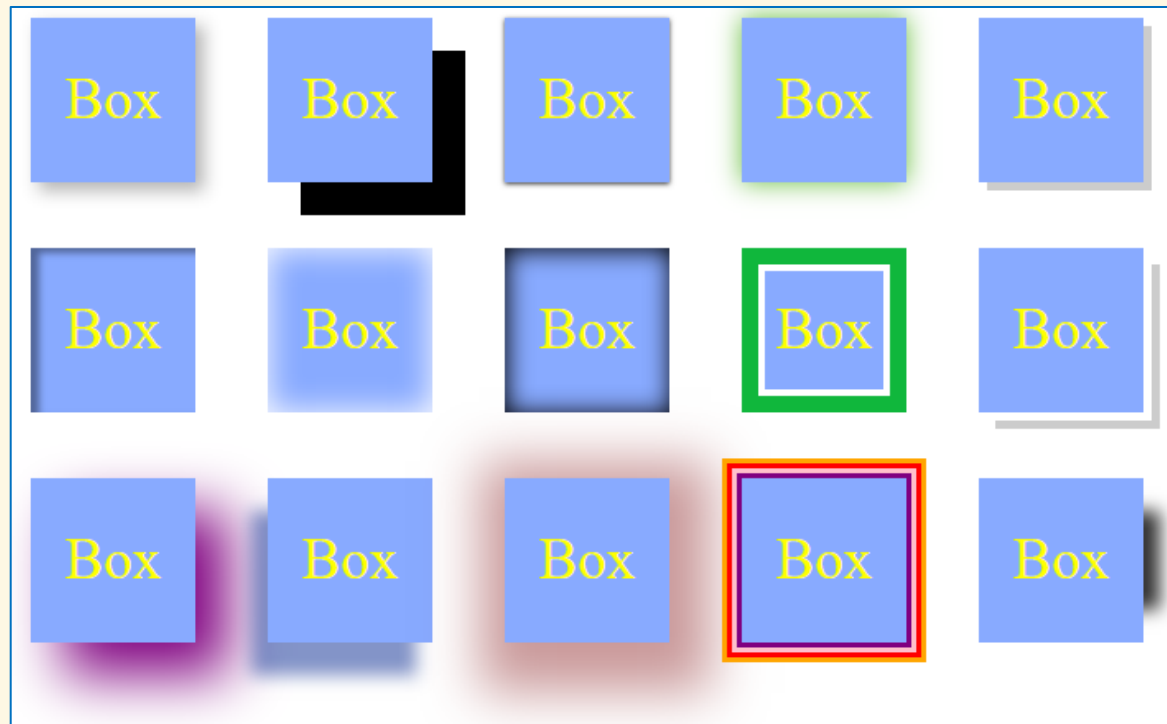
► Examples:

- `text-shadow: 2px 3px 5px gray;`
- `text-shadow: 1px 1px 1px #000, 3px 3px 5px blue;`

Box Shadow

► Syntax: `box-shadow: [inset] x y b c;`

- x = horizontal offset
 - $x < 0$: left of the box
 - $x > 0$: right of the box
- y = vertical offset
 - $y < 0$: above the box
 - $y > 0$: below the box
- b = blur radius
- c = shadow color



► Examples:

- `box-shadow: 2px 3px 5px gray;`
- `box-shadow: 1px 1px 1px #000, 3px 3px 5px blue;`

Opacity

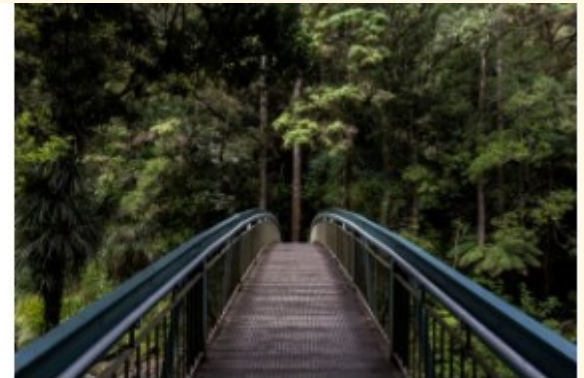
- ▶ Opacity makes the affected elements transparent



opacity: 0.2



opacity: 0.5

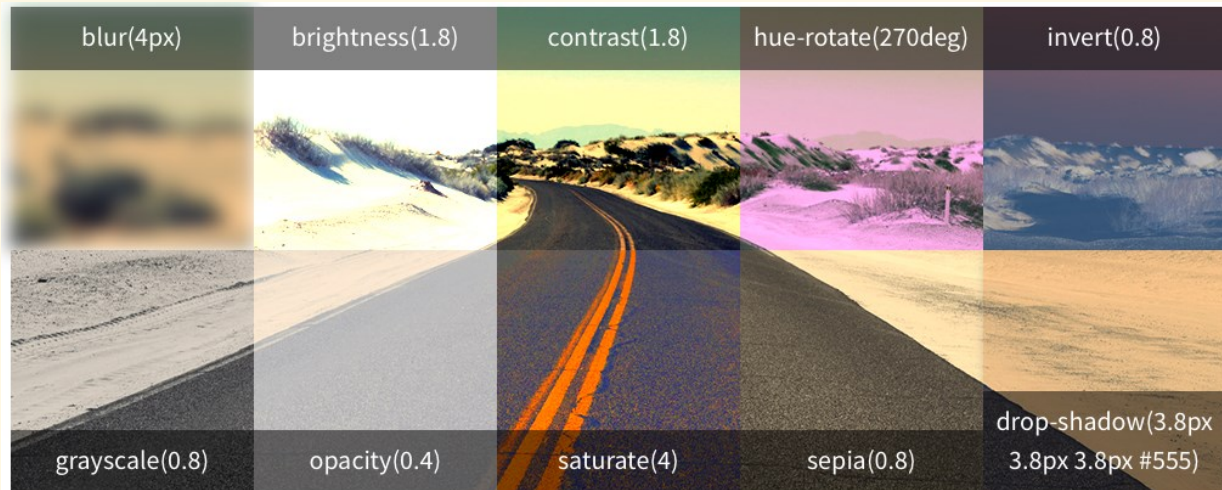


opacity: 1

Filters

▶ More effects with `filter` property:

- ▶ `opacity(0.5)`
- ▶ `blur(5px)`
- ▶ `contrast(200%)`
- ▶ `grayscale(80%)`
- ▶ `invert(70%)`
- ▶ `saturate(30%)`
- ▶ `sepia(60%)`
- ▶ `brightness(0.4)`
- ▶ `hue-rotate(90deg)`
- ▶ `drop-shadow(2px 3px 5px red)`



▶ Examples:

- ▶ `filter: blur(5px);`
- ▶ `filter: contrast(175%) blur(3px) grayscale(30%);`

Cursor

- Specifies the cursor type to be shown when user moves the pointer over the element
 - `cursor: default | none | context-menu | help | pointer | progress | wait | cell | crosshair | text | vertical-text | alias | copy | move | no-drop | not-allowed | grab | grabbing | e-resize | n-resize | ne-resize | nw-resize | s-resize | se-resize | sw-resize | w-resize | ew-resize | ns-resize | nesw-resize | nwse-resize | col-resize | row-resize | all-scroll | zoom-in | zoom-out | url(cursor-image-file)`

 auto	 move	 no-drop	 col-resize
 all-scroll	 pointer	 not-allowed	 row-resize
 crosshair	 progress	 e-resize	 ne-resize
 default	 text	 n-resize	 nw-resize
 help	 vertical-text	 s-resize	 se-resize
 inherit	 wait	 w-resize	 sw-resize

Exercise

- ▶ Create a button with shadow and pressed effect on mouse click



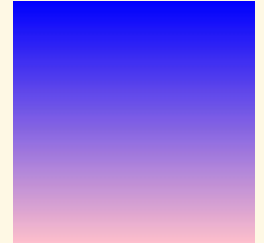
Gradients

Gradients

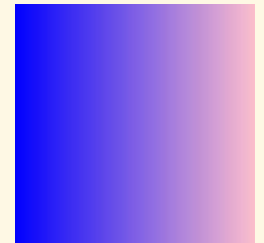
- ▶ Gradients can be used in place of images, which can be:
 - ▶ `background-image, border-image-source, list-style-image, cursor`
- ▶ Types:
 - ▶ Linear
 - ▶ Radial
 - ▶ Conic
- ▶ Examples:
 - ▶ `background-image: linear-gradient(red, #2f8, orange);`
 - ▶ `border-image-source: radial-gradient(#f88, green);`
 - ▶ `list-style-image: conic-gradient(violet, blue);`

Linear Gradients: Direction

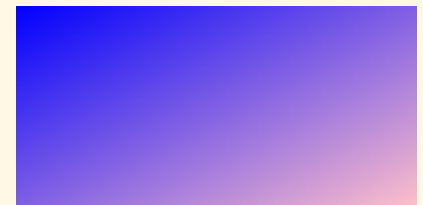
▶ `background: linear-gradient(blue, pink);`



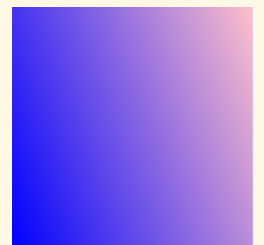
▶ `background: linear-gradient(to right, blue, pink);`



▶ `background: linear-gradient(to bottom right, blue, pink);`

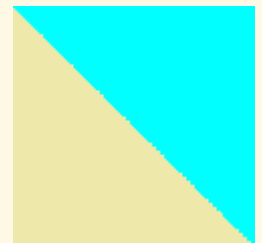
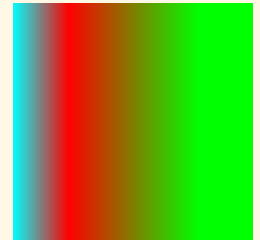
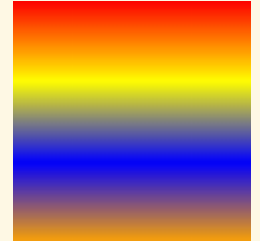


▶ `background: linear-gradient(70deg, blue, pink);`



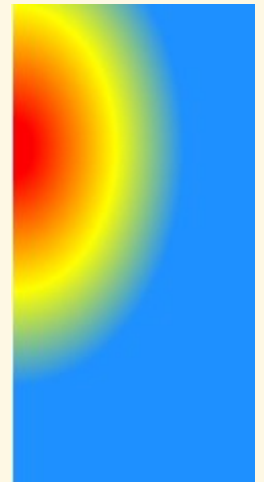
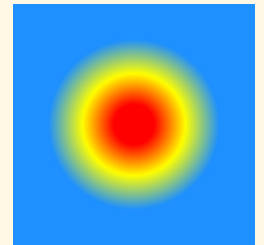
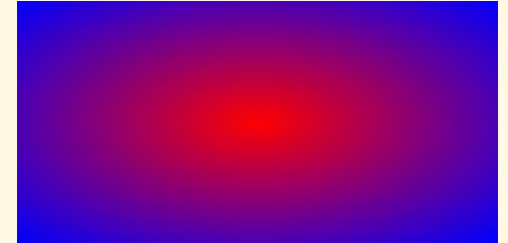
Linear Gradients: Color Stops

- ▶ `background: linear-gradient(red, yellow, blue, orange);`
- ▶ `background: linear-gradient(to left, lime 28px, red 77%, cyan);`
- ▶ `background: linear-gradient(to bottom left, cyan 50%, palegoldenrod 50%);`



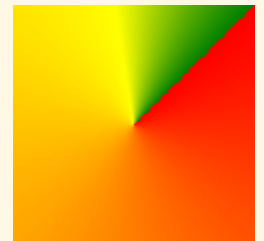
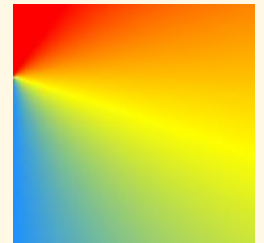
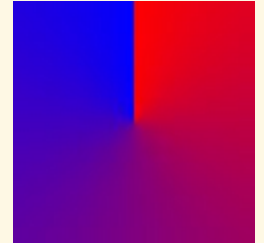
Radial Gradients

- ▶ `background: radial-gradient(red, blue);`
- ▶ `background: radial-gradient(red 10px, yellow 30%, #1e90ff 50%);`
- ▶ `background: radial-gradient(at 0% 30%, red 10px, yellow 30%, #1e90ff 50%);`



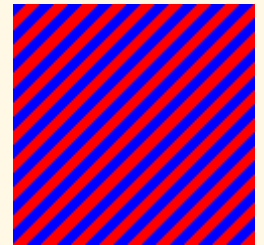
Conic Gradients

- ▶ `background: conic-gradient(red, blue);`
- ▶ `background: conic-gradient(at 0% 30%, red 10%, yellow 30%, #1e90ff 50%);`
- ▶ `background: conic-gradient(from 45deg, red, orange 50%, yellow 85%, green);`



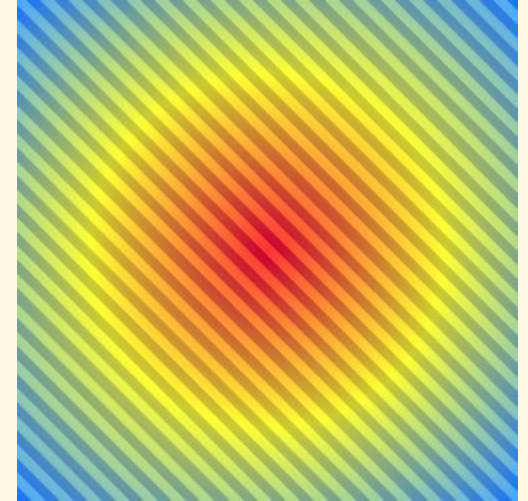
Repeating Gradients

- ▶ Use `repeating-linear-gradient()`, `repeating-radial-gradient()`, and `repeating-conic-gradient()` instead
- ▶ `background: repeating-linear-gradient(-45deg, red, red 5px, blue 5px, blue 10px);`
- ▶ `background: repeating-radial-gradient(#e66465, #9198e5 20%);`



Multiple Gradients

```
▶ background:
    radial-gradient(
        rgba(255, 0, 0, 0.8),
        rgba(255, 255, 0, 0.8),
        rgba(30, 144, 255, 0.8)),
    repeating-linear-gradient(45deg,
        rgba(0, 0, 255, 1),
        rgba(0, 0, 255, 1) 5px,
        rgba(255, 255, 255, 0) 5px,
        rgba(255, 255, 255, 0) 10px);
```



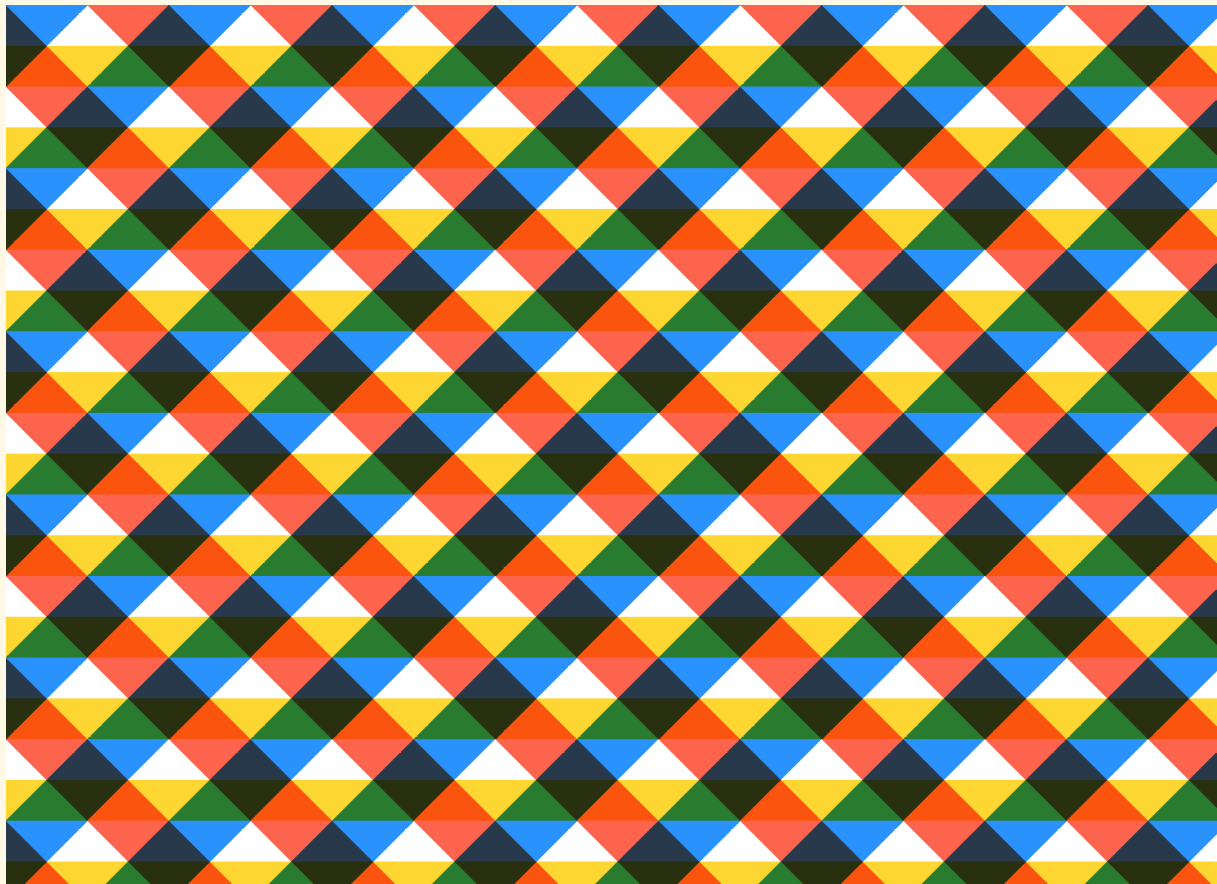
Text Gradients

- ▶ Currently CSS has no properties for text gradients
- ▶ Use the same trick as for image in text background:
 - ▶ `.text {
 color: transparent;
 background-image: linear-gradient(
 to right, red, blue, green);
 -webkit-background-clip: text;
}`

Text with Gradient

Exercise

- ▶ Create a background with gradients as in the following

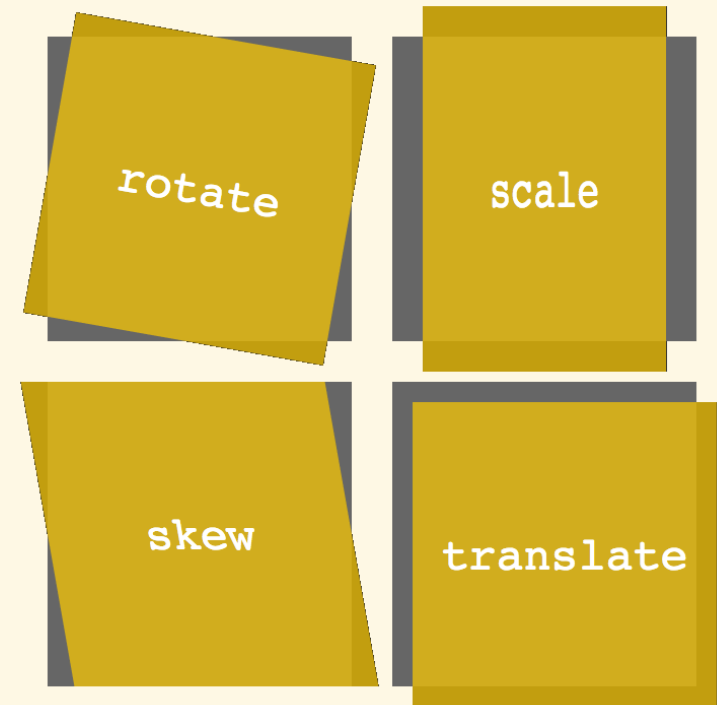


Transformations

2D Transformations

► Use `transform` property with following methods

- `translate(dx, dy)`
- `translateX(dx)`
- `translateY(dy)`
- `rotate(ang)`
- `scale(sx, sy)`
- `scaleX(sx)`
- `scaleY(sy)`
- `skew(angX, angY)`
- `skewX(ang)`
- `skewY(ang)`
- `matrix(m11, m12, m21, m22, dx, dy)`



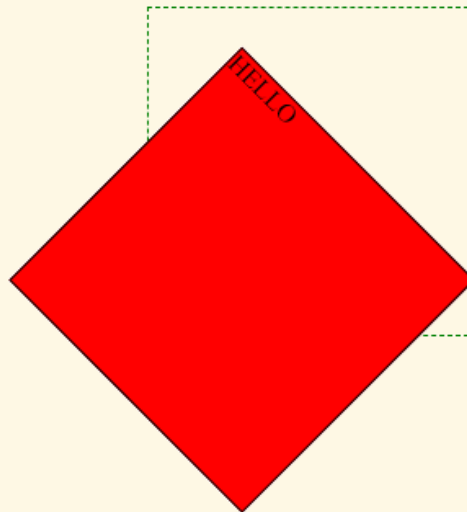
► `transform-origin`: changes the position on transformed elements

Examples

- ▶ `transform: rotate(45deg) scale(2);`



- ▶ `transform: rotate(45deg);`
`transform-origin: 0 40%;`



3D Transformations

- ▶ Use `transform` property with following methods
 - ▶ `translate3d(dx, dy, dz)`
 - ▶ `rotate3d(x, y, z, a)`
 - ▶ `scale3d(sx, sy, sz)`
 - ▶ `perspective(d)`
 - ▶ `matrix3d(...)`



Transitions

Transitions

- ▶ Effects that let an element gradually change from one style to another
- ▶ Example
 - ▶

```
div {  
    width: 200px;  
    transition: width 2s;  
}
```



```
div:hover {  
    width: 300px;  
}
```
 - ▶ Multiple transitions at once:
 - ▶

```
transition: width 2s, height 1s, transform 1.5s;
```

Detailed Properties and Shorthand

► Detailed:

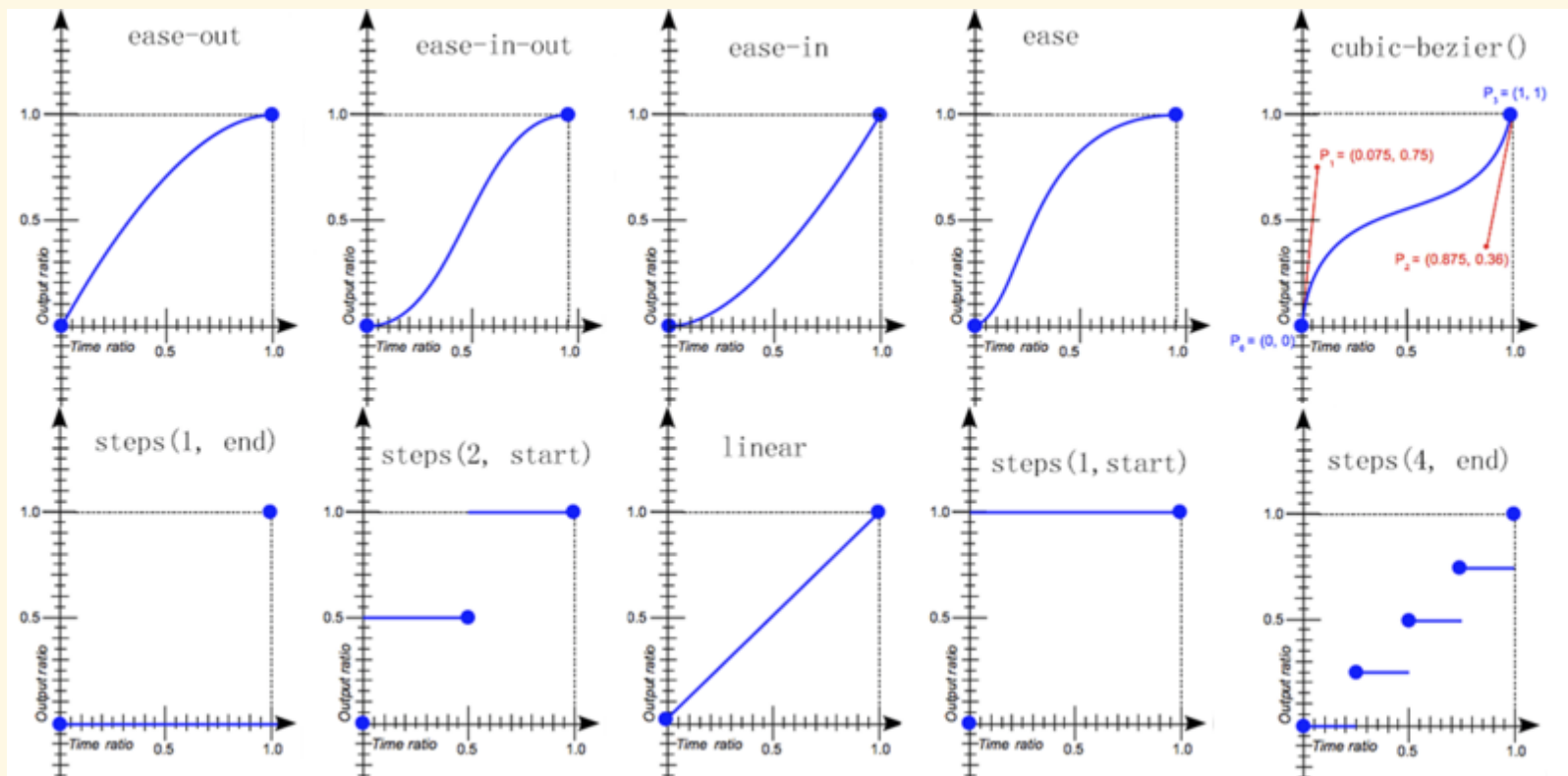
- `transition-property: width;`
`transition-duration: 1s;`
`transition-timing-function: linear;`
`transition-delay: 2s;`

► Equivalent shorthand:

- `transition: width 1s linear 2s;`

transition-timing-function

- ▶ linear|ease|ease-in|ease-out|ease-in-out
- ▶ steps(steps, start|end)
- ▶ cubic-bezier(*n*, *n*, *n*, *n*)



Animatable Properties

- ▶ Not all properties can be animated
- ▶ In general, the animatable properties are related to:
 - ▶ Color
 - ▶ Background
 - ▶ Border
 - ▶ Outline
 - ▶ Position
 - ▶ Margin, padding, size
 - ▶ Text
 - ▶ Opacity
 - ▶ Transforms, filters

Exercises

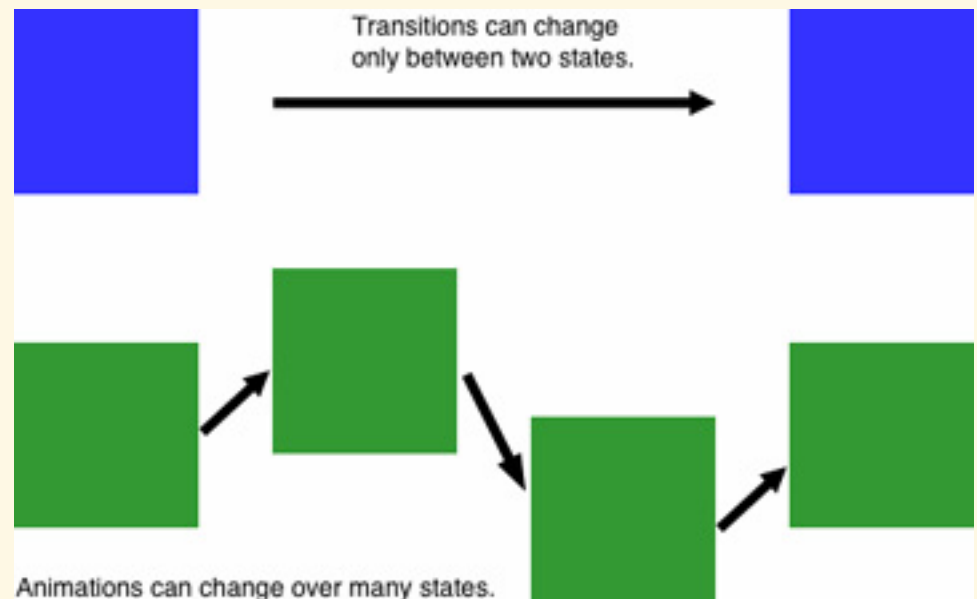
- ▶ Create a button that changes its background color, text color, border with transitions on mouse events



Animations

Introduction

- ▶ Animations let an element gradually change from one style to another
 - ▶ Possible to change multiple CSS properties, multiple times
- ▶ Animations vs transitions
 - ▶ Transitions require a trigger (hover, focus,...) to run; animations don't
 - ▶ Transitions are limited to an initial and final state; animation can include many intermediate states (called keyframes)
 - ▶ Animations can loop, be started, stopped, paused dynamically



Keyframes

- ▶ Keyframes hold the styles the element has at certain times

- ▶ Example

```
▶ @keyframes example {  
    from {background-color: red;}  
    to {background-color: yellow;}  
}
```

```
div {  
    animation-name: example;  
    animation-duration: 4s;  
}
```

Timing

- ▶ `@keyframes example {`
 - `0% {background-color: red; }`
 - `25% {background-color: yellow; }`
 - `50% {background-color: blue; }`
 - `100% {background-color: green; }``}`
- ▶ **Attn:**
 - ▶ `from = 0%`
 - ▶ `to = 100%`

Play Options

- ▶ **Delayed animation:**
 - ▶ `animation-delay: 2s;`
 - ▶ If using negative value, the animation will start as if it had already been playing a while
- ▶ **Repeated animation:**
 - ▶ `animation-iteration-count: 3;`
 - ▶ `animation-iteration-count: infinite;`
- ▶ **Animation play direction:**
 - ▶ `animation-direction: normal;` (forwards - default)
 - ▶ `animation-direction: reverse;` (backwards)
 - ▶ `animation-direction: alternate;` (fwds then bwds)
 - ▶ `animation-direction: alternate-reverse;` (bwds then fwds)
- ▶ **Timing functions**
 - ▶ `animation-timing-function` is used similarly to `transition-timing-function`

Animation Shorthand

▶ Shorthand

- ▶ `animation: example 5s linear 2s infinite alternate;`

▶ Equivalence

- ▶ `animation-name: example;`
`animation-duration: 5s;`
`animation-timing-function: linear;`
`animation-delay: 2s;`
`animation-iteration-count: infinite;`
`animation-direction: alternate;`

Pausing and Resuming an Animation

- ▶ **Animation play state:**

- ▶ `animation-play-state: paused | running`

- ▶ **Example:**

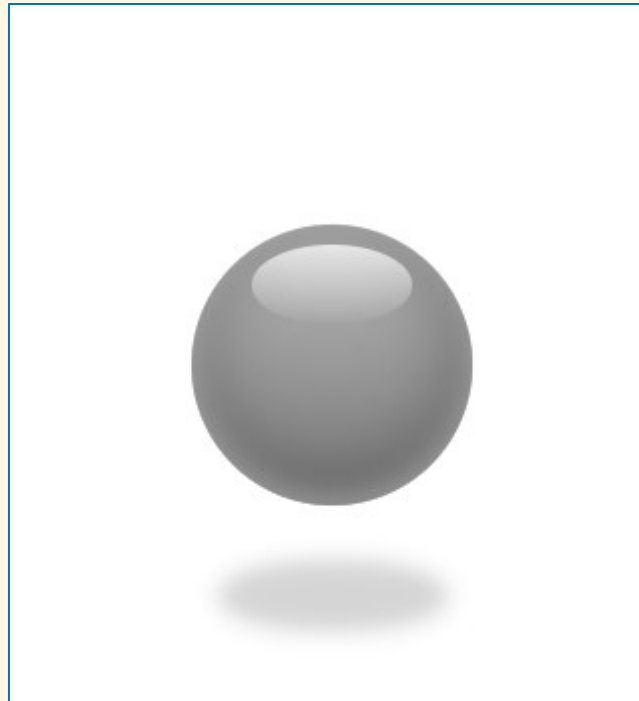
- ▶

```
div {  
    animation-name: example;  
    animation-duration: 4s;  
    animation-direction: alternate;  
    animation-iteration-count: infinite;  
}
```

```
div:hover {  
    animation-play-state: paused;  
}
```

Exercise

- ▶ Create and style a bouncing ball with animation as in the following:



Media Queries and Responsive Design

Motivation

- ▶ Document should look good on any device (desktop, tablet, phone) → responsive (dynamic) layouts
- ▶ Solution: Use CSS to shrink, enlarge, hide or move content in order to look good on any screen
- ▶ How to do:
 - ▶ Setting viewport:
 - ▶ `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
 - ▶ Do not rely on a particular viewpoint
 - ▶ Use relative dimensions (%): width, height
 - ▶ Use relative units (em, rem)
 - ▶ Use flexbox for layout
 - ▶ Use media-queries to apply different styles to large/small screens

Media Queries

- ▶ Use @media to apply CSS style only if a condition is met
 - ▶ @media only screen and (max-width:500px) {
 #div1 {
 width: 100%;
 }
}
 - ▶ @media only screen and (min-width:500px) { }
 - ▶ @media only screen and (orientation:landscape) { }

Media Query Example

```
▶ body { display: flex; }  
#left { width: 25%; }  
#main { width: 75%; }
```

```
@media screen and (max-width:  
800px) {  
  body {  
    flex-direction: column;  
  }  
  #left, #main {  
    width: 100%;  
  }  
}
```

```
▶ <section id="left">...</section>  
<section id="main">...</section>
```

Item #1
Item #2
Item #3
Item #4

The main section

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vitae dui at eros dictum finibus vel eget leo. Nam feugiat pharetra orci, eget ultricies erat imperdiet eu. Vivamus convallis, nunc eget imperdiet scelerisque, eros elit lobortis sapien, id mattis enim felis sit amet turpis. Nunc in massa dolor. Pellentesque elit purus, laoreet sit amet cursus eget, pharetra sed enim. In hac habitasse platea dictumst. In porttitor eleifend lacus, sit amet consectetur massa convallis eu.

Praesent semper sapien mi, quis mollis tellus rutrum id. Nullam fringilla elementum gravida. Duis diam massa, hendrerit in dapibus vitae, semper id est. Quisque tempus nibh vehicula, porttitor est et, tempus metus. Praesent sed euismod dolor. Integer suscipit luctus mi a condimentum. Praesent pulvinar, magna vel feugiat viverra, ex felis lobortis diam, in gravida nulla nisi sed purus. Ut vestibulum porttitor tristique. Mauris vel quam vel risus imperdiet tristique. Aenean vehicula iaculis leo at bibendum. Curabitur eget tincidunt tellus, et porttitor velit. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aenean aliquam turpis ac orci pharetra pharetra. Cras maximus nulla suscipit libero euismod aliquet. Duis pharetra hendrerit ex id faucibus. Donec vulputate justo id dolor mollis rutrum. Etiam iaculis velit vitae lobortis venenatis. Phasellus pharetra enim sit amet neque ultricies, vitae tristique lectus volutpat. Sed hendrerit ex quis magna venenatis porttitor. Vestibulum imperdiet metus id sem tincidunt lacinia. Duis id ornare nunc. Vivamus rutrum est lacus. Maecenas commodo eleifend sem porttitor pharetra. Morbi aliquam est sed diam interdum bibendum. Morbi vitae placerat sem, non malesuada libero. Phasellus finibus dapibus arcu, eget suscipit massa posuere vitae.

Item #1
Item #2
Item #3
Item #4

The main section

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis vitae dui at eros dictum finibus vel eget leo. Nam feugiat pharetra orci, eget ultricies erat imperdiet eu. Vivamus convallis, nunc eget imperdiet scelerisque, eros elit lobortis sapien, id mattis enim felis sit amet turpis. Nunc in massa dolor. Pellentesque elit purus, laoreet sit amet cursus eget, pharetra sed enim. In hac habitasse platea dictumst. In porttitor eleifend lacus, sit amet consectetur massa convallis eu.

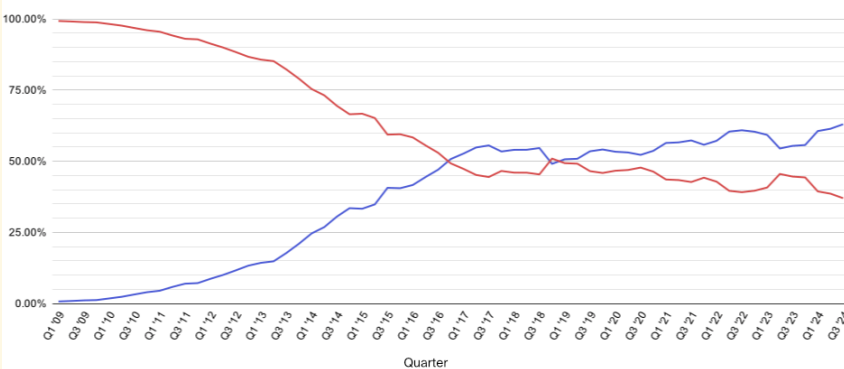
Praesent semper sapien mi, quis mollis tellus rutrum id. Nullam fringilla elementum gravida. Duis diam massa, hendrerit in dapibus vitae, semper id est. Quisque tempus nibh vehicula, porttitor est et, tempus metus. Praesent sed euismod dolor. Integer suscipit luctus mi a condimentum. Praesent pulvinar, magna vel feugiat viverra, ex felis lobortis diam, in gravida nulla nisi sed purus. Ut vestibulum porttitor tristique. Mauris vel quam vel risus imperdiet tristique. Aenean vehicula iaculis leo at bibendum. Curabitur eget tincidunt tellus, et porttitor velit. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Aenean aliquam turpis ac orci pharetra pharetra. Cras maximus nulla suscipit libero euismod aliquet. Duis pharetra hendrerit ex id faucibus. Donec vulputate justo id dolor mollis rutrum. Etiam iaculis velit vitae lobortis venenatis. Phasellus pharetra enim sit amet neque ultricies, vitae tristique lectus volutpat. Sed hendrerit ex quis magna venenatis porttitor. Vestibulum imperdiet metus id sem tincidunt lacinia. Duis id ornare nunc. Vivamus rutrum est lacus. Maecenas commodo eleifend sem porttitor pharetra. Morbi aliquam est sed diam interdum bibendum. Morbi vitae placerat sem, non malesuada libero. Phasellus finibus dapibus arcu, eget suscipit massa posuere vitae.

Mobile-First Design

- ▶ Mobile Internet use passed desktop in Oct 2016
- ▶ Mobile-first design = design for mobile before designing for desktop or any other device
 - ▶ This will make the page display faster on smaller devices
 - ▶ Ex: Instead of changing styles when the width gets **smaller** than 768px, we should change the design when the width gets **larger** than 768px

Percentage of desktop vs mobile - Worldwide



Typical Device Breakpoints

- ▶ **Extra small devices: phones ($\leq 600\text{px}$)**
 - ▶ `@media only screen and (max-width: 600px) { }`
- ▶ **Small devices: portrait tablets and large phones ($\geq 600\text{px}$)**
 - ▶ `@media only screen and (min-width: 600px) { }`
- ▶ **Medium devices: landscape tablets ($\geq 768\text{px}$)**
 - ▶ `@media only screen and (min-width: 768px) { }`
- ▶ **Large devices: laptops, desktops ($\geq 992\text{px}$)**
 - ▶ `@media only screen and (min-width: 992px) { }`
- ▶ **Extra large devices: large laptops and desktops ($\geq 1200\text{px}$)**
 - ▶ `@media only screen and (min-width: 1200px) { }`

General Syntax

► Syntax:

- `@media [not|only] mediatype and (mediafeature and|or|not mediafeature) { }`
- `mediatype: all|screen|print|speech`
- `mediafeature: width|max-width|min-width|orientation|...`

► Examples

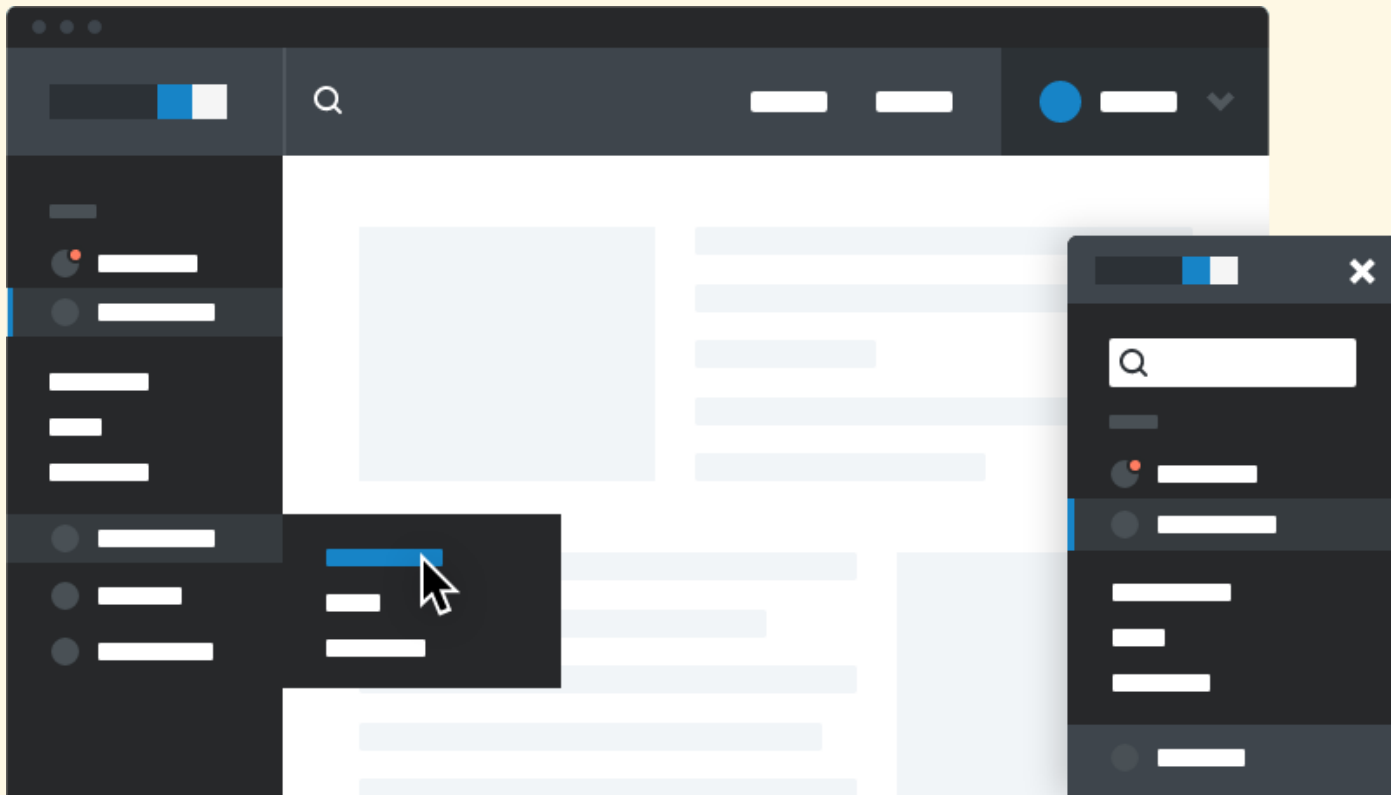
- `@media only screen and (orientation:landscape) { }`
- `@media print { }`
- `@media screen and (max-width: 900px) and (min-width: 600px), (min-width: 1100px) { }`
(When the width is between 600px and 900px, or above 1100px)

Conditional CSS Attachment

- ▶ External CSS stylesheets can be applied with conditions
 - ▶ `<link rel="stylesheet" type="text/css" href="screen.css" media="screen" />`
 - ▶ `<link rel="stylesheet" type="text/css" href="print.css" media="print" />`
 - ▶ `<link rel="stylesheet" type="text/css" href="main_1.css" media="screen and (max-device-width: 480px)" />`

Exercise

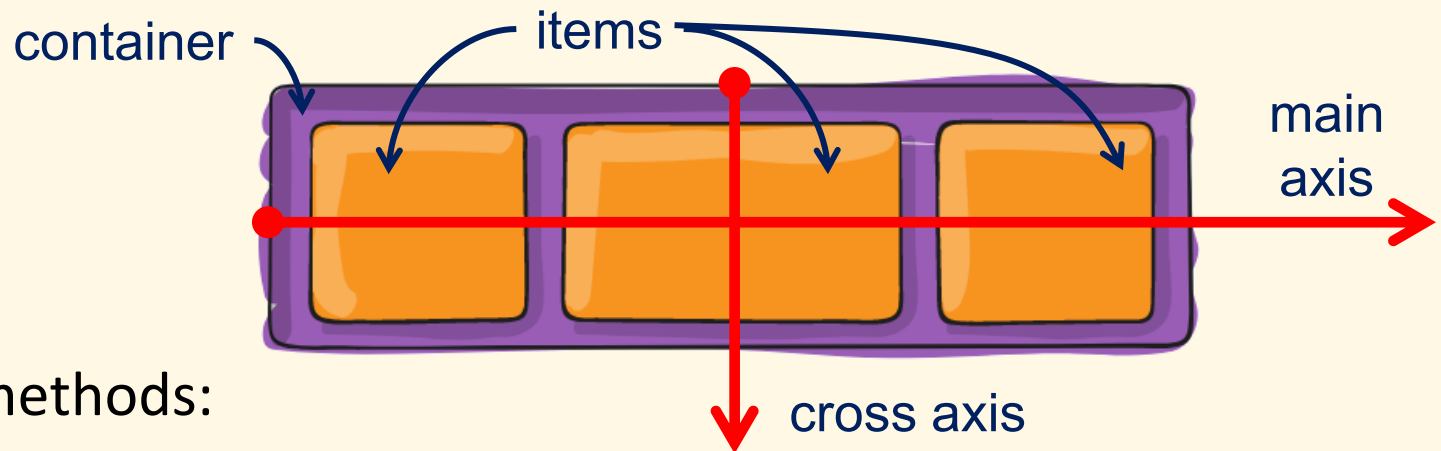
- Design the following responsive page layout with a menubar and a sidebar



Flexbox Layout

Introduction

- ▶ Flexbox (flexible box) aims at providing a more efficient way to define layouts
- ▶ More flexibility to support complex page structures
- ▶ A flexbox consists of a container (parent) and its items (children)

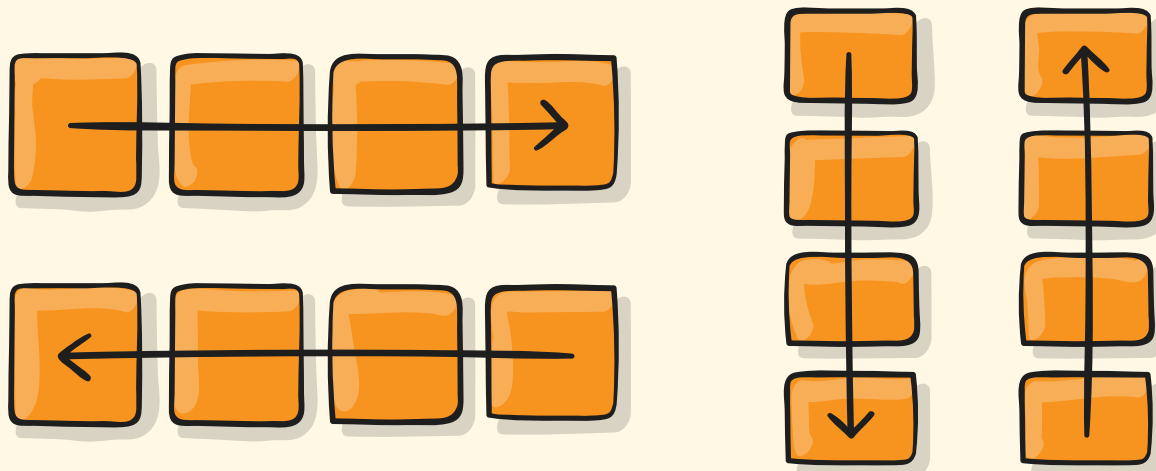


- ▶ Display methods:

```
.container {  
  display: flex; /* or inline-flex */  
}
```

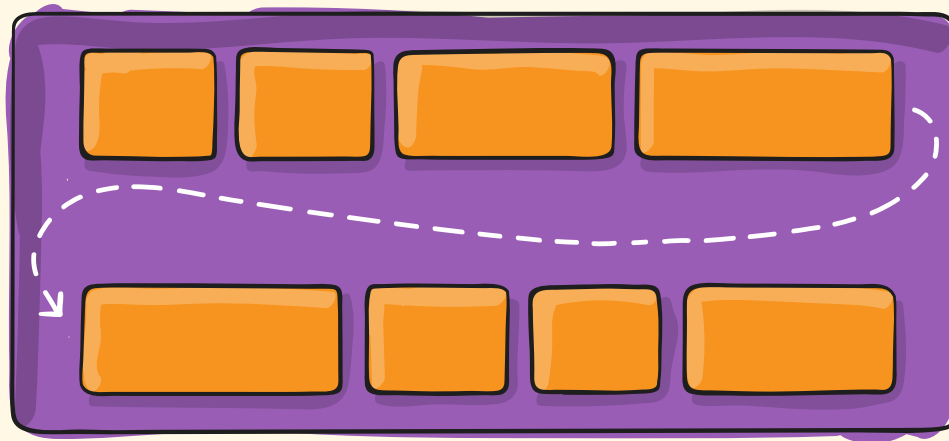
Container: `flex-direction`

- ▶ `row` (default): left to right in `ltr`; right to left in `rtl`
- ▶ `row-reverse`: right to left in `ltr`; left to right in `rtl`
- ▶ `column`: same as `row` but top to bottom
- ▶ `column-reverse`: same as `row-reverse` but bottom to top



Container: flex-wrap

- ▶ nowrap (default): all items will be on one line
- ▶ wrap: items will wrap onto multiple lines, from top to bottom
- ▶ wrap-reverse: items will wrap onto multiple lines from bottom to top

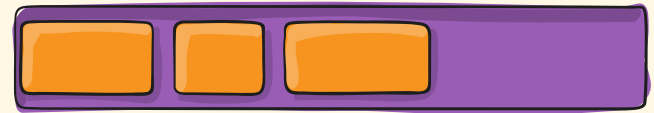


- ▶ Shorthand for flex-direction and flex-wrap:
 - ▶ flex-flow: column-reverse wrap;

Container: justify-content

- ▶ flex-start (default), flex-end: items packed toward the start/end of the flex direction
- ▶ start, end: same as flex-start/flex-end, but wrt the writing-mode direction
- ▶ left, right, center: items are packed toward the left/right/center of the container
- ▶ space-between, space-around, space-evenly: items evenly distributed in the line

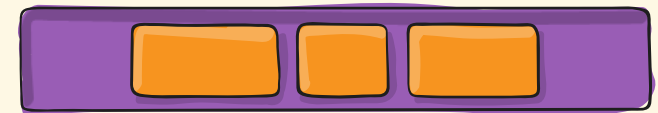
flex-start



flex-end



center



space-between



space-around



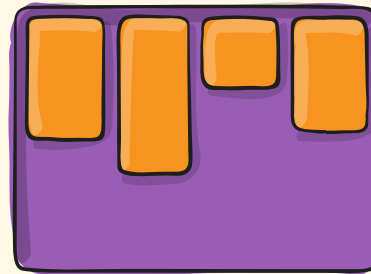
space-evenly



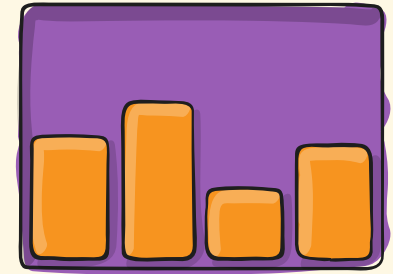
Container: align-items

- ▶ stretch (default): items stretch to fill the container (still respect min-width/max-width)
- ▶ flex-start, start, self-start: items placed at the start of the cross axis. The difference of them is about respecting the flex-direction or the writing-mode rules
- ▶ flex-end, end, self-end: items placed at the end of the cross axis
- ▶ center: items centered in the cross-axis
- ▶ baseline: items aligned such as their baselines align

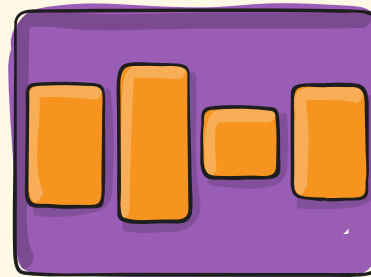
flex-start



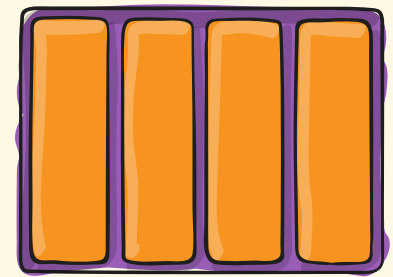
flex-end



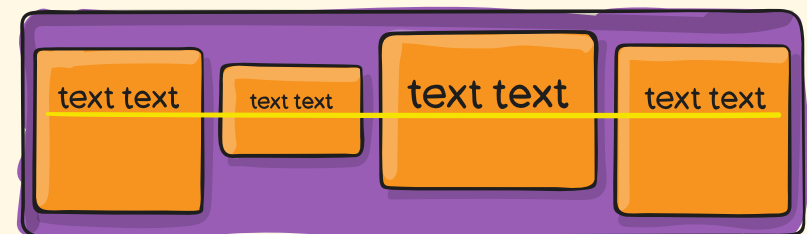
center



stretch



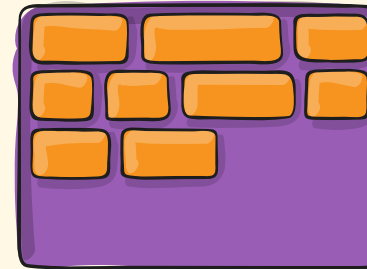
baseline



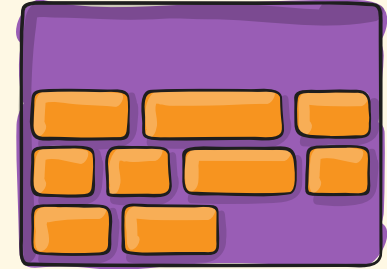
Container: align-content

- ▶ `normal` (default): items packed in their default position
- ▶ `flex-start`, `start`: items packed to the start of the container, `flex-start` wrt the `flex-direction`, while `start` wrt the writing-mode direction
- ▶ `flex-end`, `end`: items packed to the end of the container
- ▶ `center`: items centered in the container
- ▶ `stretch`: lines stretch to take up the remaining space
- ▶ `space-between`, `space-around`, `space-evenly`: items evenly distributed

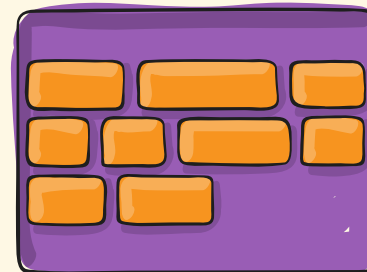
`flex-start`



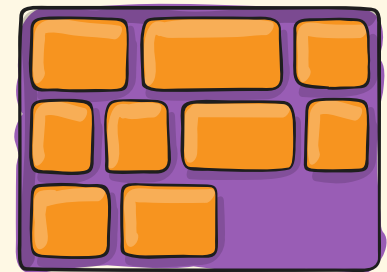
`flex-end`



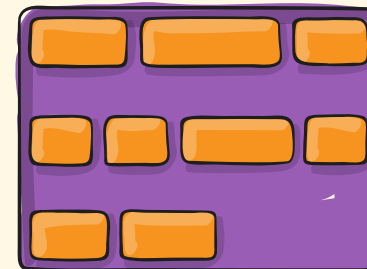
`center`



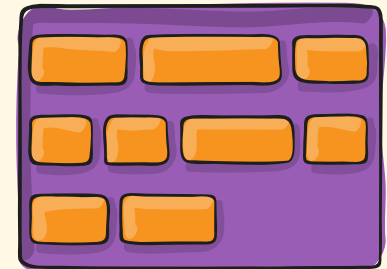
`stretch`



`space-between`



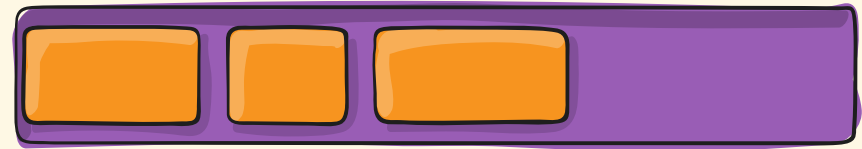
`space-around`



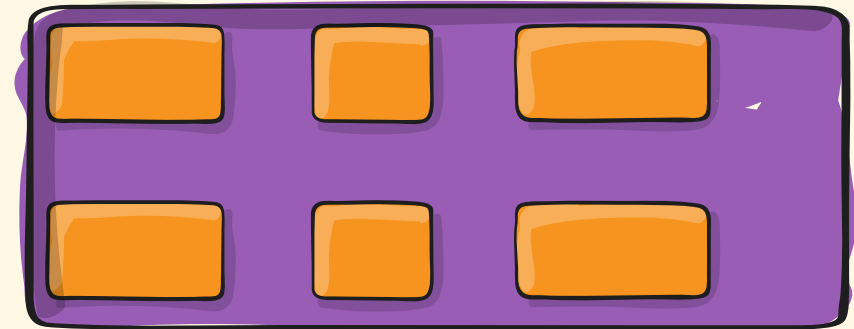
Container: gap, row-gap, column-gap

- ▶ To control the gap between rows and columns
- ▶ Examples:
 - ▶ `gap: 10px`
 - ▶ `gap: 5% 0.5rem`
 - ▶ `row-gap: 2em`
 - ▶ `column-gap: 10vh`

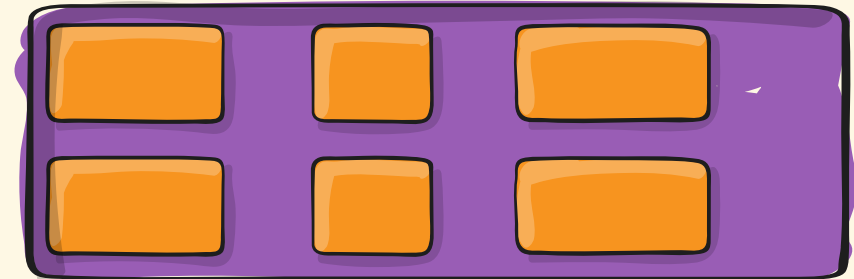
gap: 10px



gap: 30px



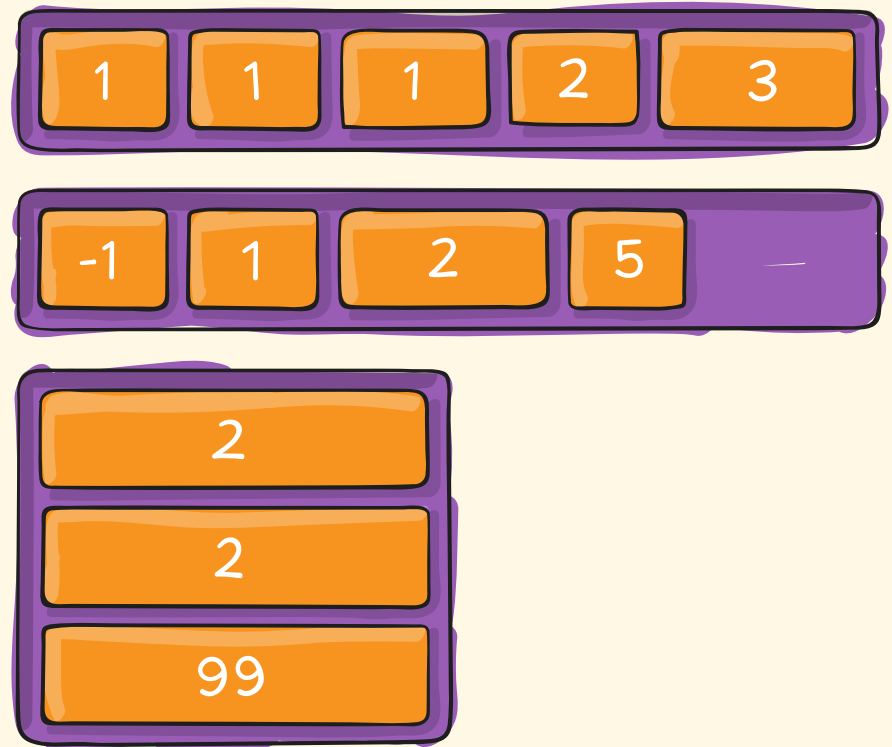
gap: 10px 30px



Items: order

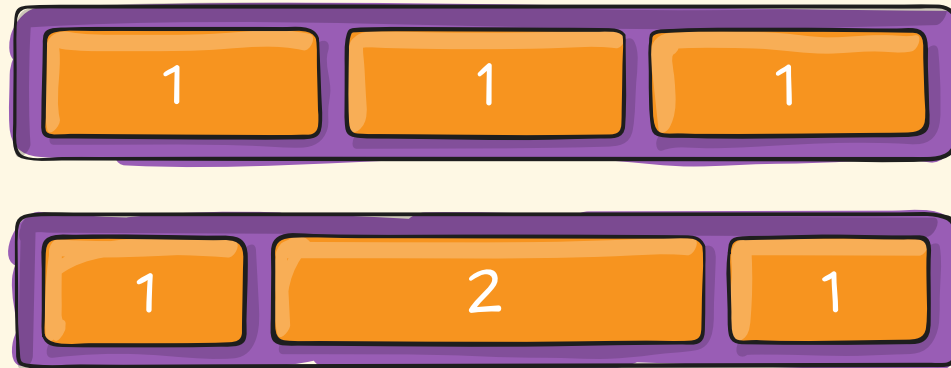
- ▶ Helps to alternate the default order
- ▶ Items with same value are laid out in their source order
- ▶ Default value: 0
- ▶

```
.item {  
    order: 5;  
}
```



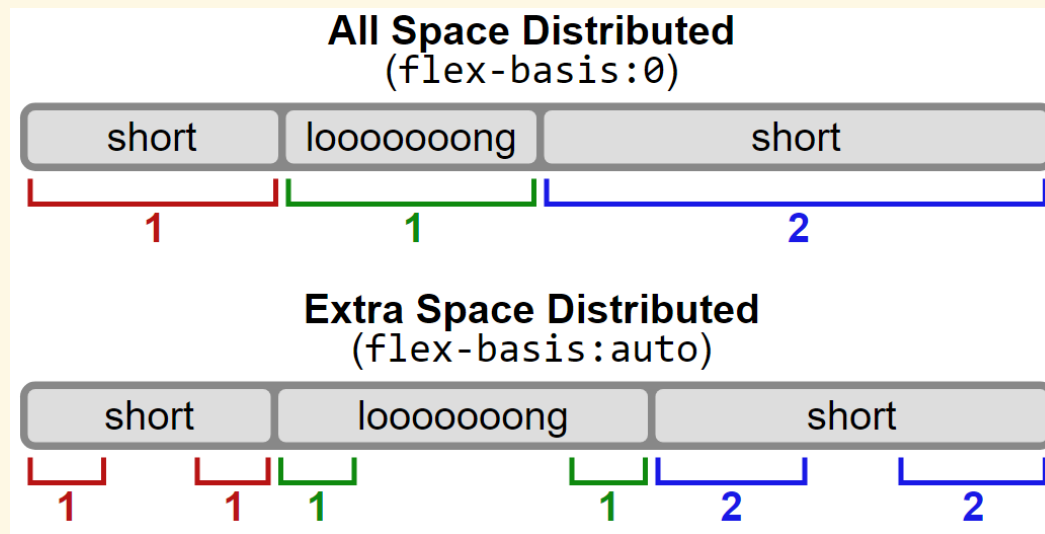
Items: `flex-grow`

- ▶ Defines the ability for an item to grow if necessary: how much the remaining space is distributed to each item
- ▶ Value serves as a proportion: an item set to 2 would take up twice as much as space as an item set to 1
- ▶ Default value: 0



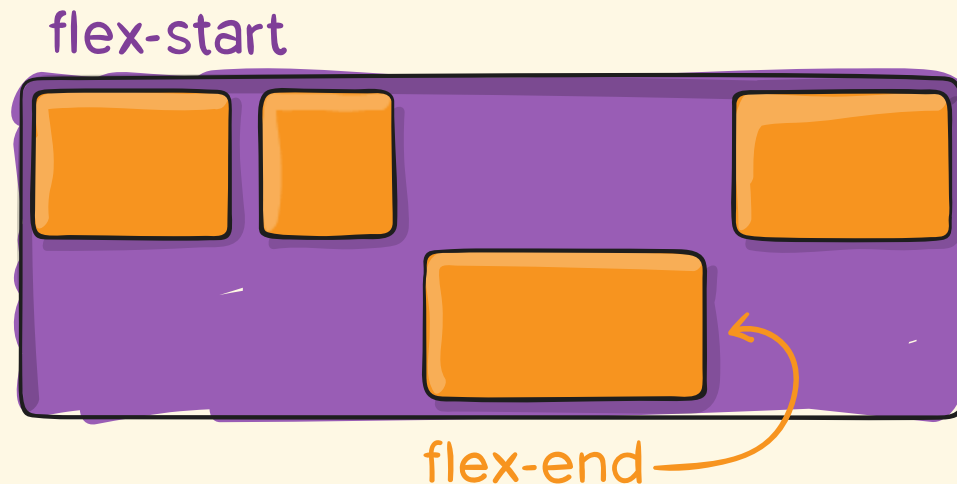
Items: flex-basis

- ▶ Defines the default size of an element before the remaining space is distributed
 - ▶ Can be a length: 20%, 100px, 10rem
 - ▶ 0: the extra space around content isn't factored in
 - ▶ auto: the extra space is distributed based on its flex-grow value



Items: `flex-self`

- ▶ Allows the default alignment (or the one specified by container's `align-items`) to be overridden for individual flex items
- ▶ See `align-items` for the values



Two-Value display Property

- ▶ Refactored into outer and inner types → more generalized way

- ▶ Outer type:

- ▶ block
- ▶ inline

- ▶ Inner type:

- ▶ flow
- ▶ flow-root
- ▶ flex
- ▶ grid
- ▶ table

Legacy (precomposed)	New (two values)
block	block flow
flow-root	block flow-root
inline	inline flow
inline-block	inline flow-root
flex	block flex
inline-flex	inline flex
grid	block grid
inline-grid	inline grid
table	block table
inline-table	inline table

Multi-column Layout

Introduction

- ▶ Allows to define multi-column layouts
- ▶ Basic example
 - ▶

```
.container {  
    column-count: 3;  
}
```

Lorem ipsum

dolor sit amet, consectetur
adipiscing elit, sed diam
nonummy nibh euismod tincidunt
ut laoreet dolore magna aliquam
erat volutpat. Ut wisi enim ad
minim veniam, quis nostrud

exercitation ullamcorper suscipit
lobortis nisl ut aliquip ex ea
commodo consequat. Duis autem
vel eum iriure dolor in hendrerit
in vulputate velit esse molestie
consequat, vel illum dolore eu
feugiat nulla facilisis at vero eros
et accumsan et iusto odio

dignissim qui blandit praesent
luptatum zzril delenit augue duis
dolore te feugait nulla facilisi.
Nam liber tempor cum soluta
nobis eleifend option congue nihil
imperdiet doming id quod mazim
placera facer possim assum.

Layout Options

- ▶ **Minimum width of columns:**
 - ▶ `column-width: 100px;`
- ▶ **Shorthand for `column-width` and `column-count`:**
 - ▶ `columns: 100px 3;`
- ▶ **Gap between columns**
 - ▶ `column-gap: 40px;`
- ▶ **Style of the rule between columns**
 - ▶ `column-rule: 2px solid red;`
 - ▶ `column-rule-style: solid|dotted|dashed|...;`
 - ▶ `column-rule-color: red;`
 - ▶ `column-rule-width: 2px;`
- ▶ **Column filling mode:**
 - ▶ `column-fill: balance|auto;`

Element Span

► Number of columns an element should span across

► `column-span: all | none;`

London. Michaelmas term lately over, and the Lord Chancellor sitting in Lincoln's Inn Hall.	effect of column- span property.	earth, and it would not be wonderful to meet a Megalosaurus, forty feet long or so, waddling like an elephantine lizard up Holborn Hill.
Spanner? This element is to show the	Implacable November weather. As much mud in the streets as if the waters had but newly retired from the face of the	

`column-span: none`

London. Michaelmas term lately	over, and the Lord Chancellor	sitting in Lincoln's Inn Hall.
Spanner? This element is to show the effect of column-span property.		
Implacable November weather. As much mud in the streets as if the waters had but newly	retired from the face of the earth, and it would not be wonderful to meet a Megalosaurus,	forty feet long or so, waddling like an elephantine lizard up Holborn Hill.

`column-span: all`

More Features

CSS Variables (aka. Custom Properties)

- ▶ Entities that contain specific values to be reused throughout a document
- ▶ Accessible using `var (...)`
- ▶ Case-sensitive
- ▶ Example:
 - ▶

```
element {  
    --main-color: brown;  
    background-color: var(--main-color);  
}
```

Variables in `:root` pseudo-class

- ▶ In order to make the variables anywhere throughout the document

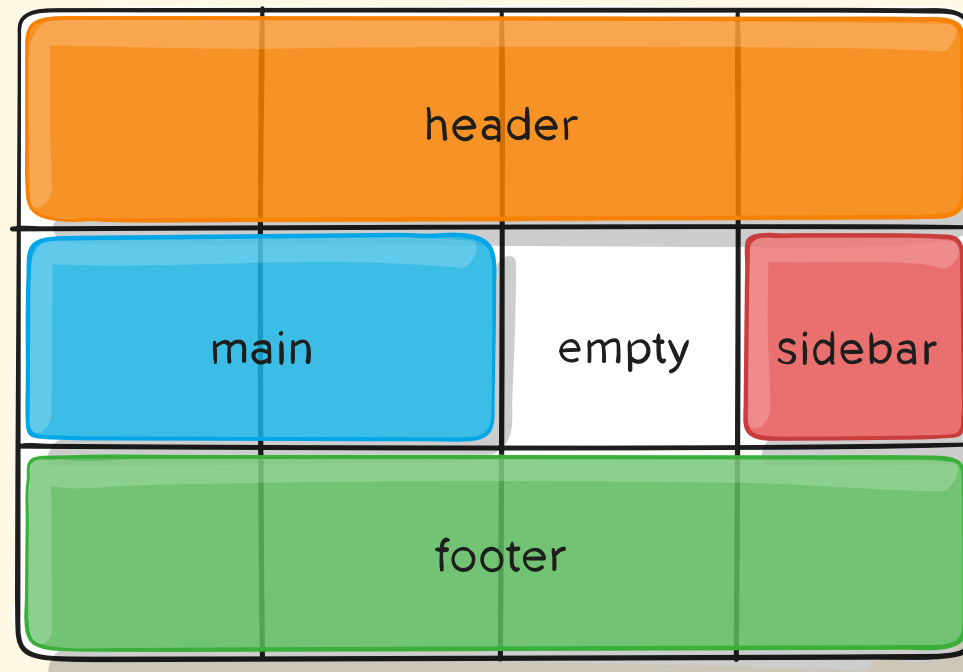
```
▶ :root {  
    --color: brown;  
    --size: 10px;  
}
```

```
.one {  
    background-color: var(--color);  
}
```

```
.two {  
    --size-2: calc(var(--size) + 5px);  
    width: calc(var(--size) * 2);  
}
```

Grid Layout

- ▶ A 2D grid-based layout system that helps to design grid-based layouts
- ▶ To be self-studied



Even More

- ▶ Container queries
- ▶ CSS nesting
- ▶ Scroll driven animations
- ▶ View transitions
- ▶ ...